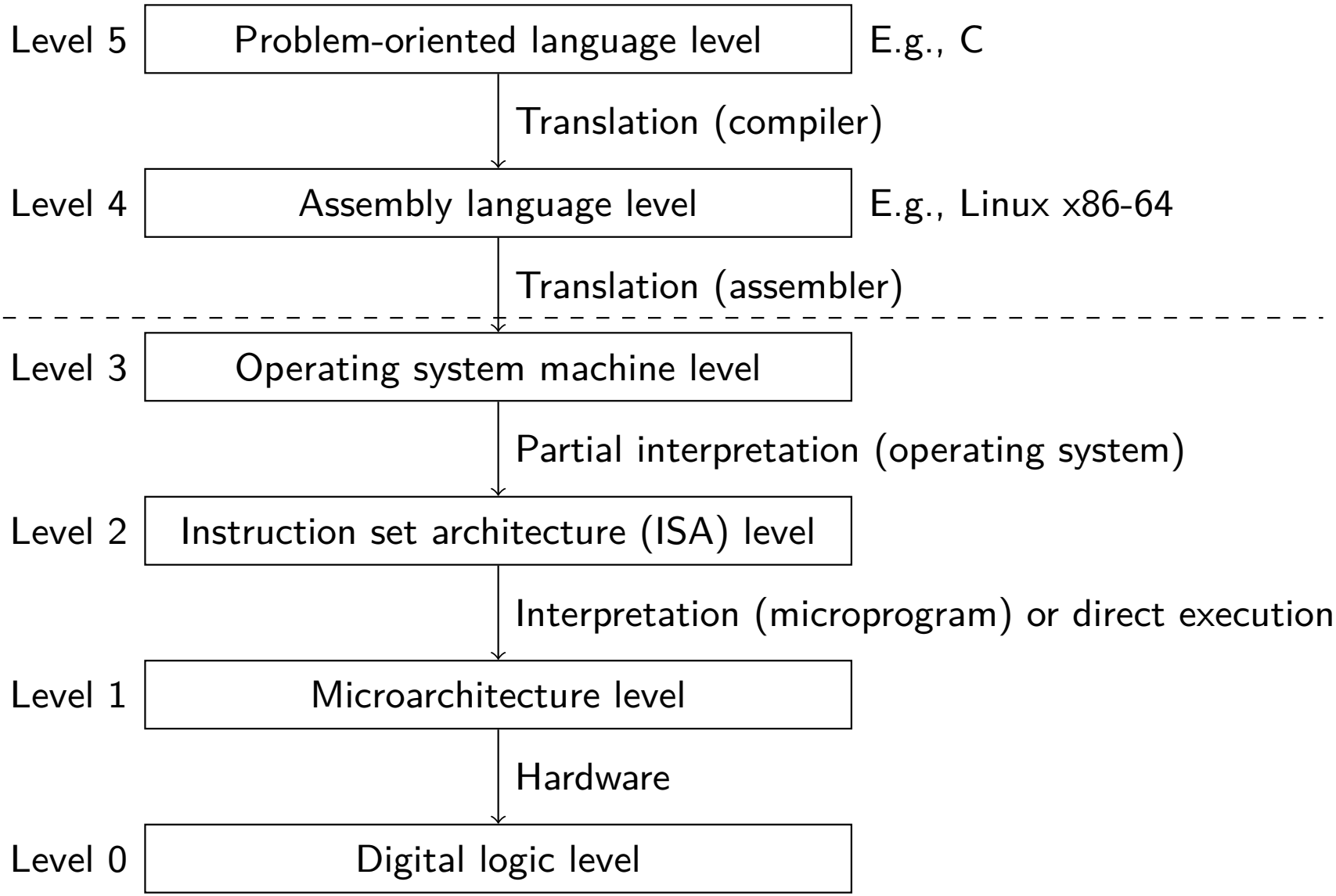
DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2025

Digital Logic

- Boolean Algebra
- Gates and Circuits
- Combinational Circuits
- Minimizing Circuits
- Important Combinational Circuits
- Sequential Circuits

Where Are We?


Level 5
Problem-oriented language level
E.g., C
Translation (compiler)
Level 4
Assembly language level
E.g., Linux x86-64
Translation (assembler)
Level 3
Operating system machine level
Partial interpretation (operating system)
Level 2
Instruction set architecture (ISA) level
Interpretation (microprogram) or direct execution
Level 1
Microarchitecture level
Hardware
Level 0
Digital logic level

- Boolean Algebra
- Gates and Circuits
- Combinational Circuits
- Minimizing Circuits
- Important Combinational Circuits
- Sequential Circuits

Boolean Algebra

- Named after George Boole, who proposed the basics in 1854.
- Claude Shannon suggested in 1938 that it may be useful for solving problems in relay-switching circuit design.
- A convenient mathematical language for describing the function of digital circuitry,
- and for developing a simplified implementation of that function.

Variables and Operations

- A boolean variable may have two values: 0 (FALSE) or 1 (TRUE).
- Basic operations:
  - AND (conjunction)
    - The result is 1 if and only if all inputs are 1.
    - AND has precedence over OR.
    - Different writing styles: A AND B, $A \wedge B$, $A \cdot B$, or simply AB
  - OR (disjunction)
    - The result is 1 if and only if at least one input is 1.
    - Different writing styles: A OR B, $A \vee B$, $A + B$
  - NOT (negation)
    - The result is 1 if and only if the input is 0.
    - Different writing styles: NOT A, $\neg A$, $\overline{A}$, A'
- Other operations: NAND ("NOT AND"), NOR ("NOT OR"), XOR (exclusive OR, $\oplus$)
- (Be careful with notation in larger expressions)

Boolean Operators and Truth Tables

Truth table:

- A column for each input and each output.
- A row for each combination of the inputs.

Two input variables:

P	Q	NOT P $\overline{P}$	P AND Q PQ	P OR Q $P+Q$	P NAND Q $\overline{PQ}$	P NOR Q $\overline{P+Q}$	P XOR Q $P \oplus Q$
0	0	1	0	0	1	1	0
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	1
1	1	0	1	1	0	0	0

Extended to more than two inputs, A, B, ...:

Operator	Expression	Result is 1 iff
AND	$A \cdot B \cdot \ldots$	all of $A, B, \ldots$ are 1.
OR	$A + B + \ldots$	any of $A, B, \ldots$ are 1.
NAND	$\overline{A \cdot B \cdot \ldots}$	any of $A, B, \ldots$ are 0.
NOR	$\overline{A + B + \ldots}$	all of $A, B, \ldots$ are 0.
XOR	$A \oplus B \oplus \ldots$	an odd number of $A, B, \ldots$ are 1.

Basic Identities

Basic postulates:

$A \cdot B = B \cdot A$	$A + B = B + A$	Commutative laws
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Distributive laws
$1 \cdot A = A$	$0 + A = A$	Identity elements
$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$	Inverse elements

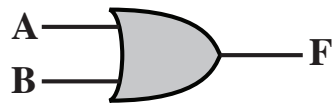
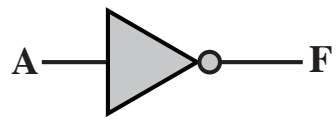
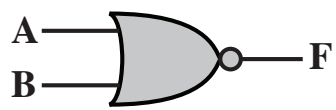
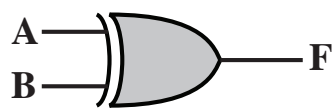
Other identities:

$0 \cdot A = 0$	$1 + A = 1$	
$A \cdot A = A$	$A + A = A$	
$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$	Associative laws
$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$	DeMorgan's laws

- Boolean Algebra
- Gates and Circuits
- Combinational Circuits
- Minimizing Circuits
- Important Combinational Circuits
- Sequential Circuits

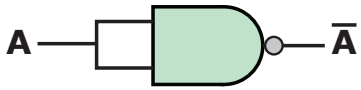
Gates

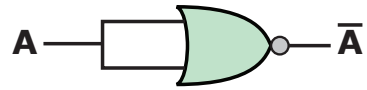
- An electronic circuit implementing one of the basic Boolean operators.
- More input lines may be drawn to each of the graphical symbols (except NOT).
- Gate delay: the time it takes for the output to be correct after an input change.

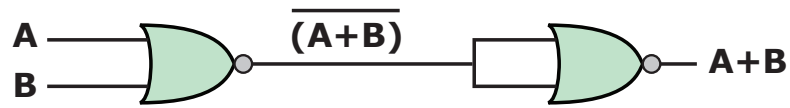
Name	Graphical Symbol	Algebraic Function	Truth Table
AND		$F = A \cdot B$ or $F = AB$	A B F 0 0 0 0 1 0 1 0 0 1 1 1
OR		$F = A + B$	A B F 0 0 0 0 1 1 1 0 1 1 1 1
NOT		$F = \overline{A}$ or $F = A'$	A F 0 1 1 0
NAND		$F = \overline{AB}$	A B F 0 0 1 0 1 1 1 0 1 1 1 0
NOR		$F = \overline{A + B}$	A B F 0 0 1 0 1 0 1 0 0 1 1 0
XOR		$F = A \oplus B$	A B F 0 0 0 0 1 1 1 0 1 1 1 0

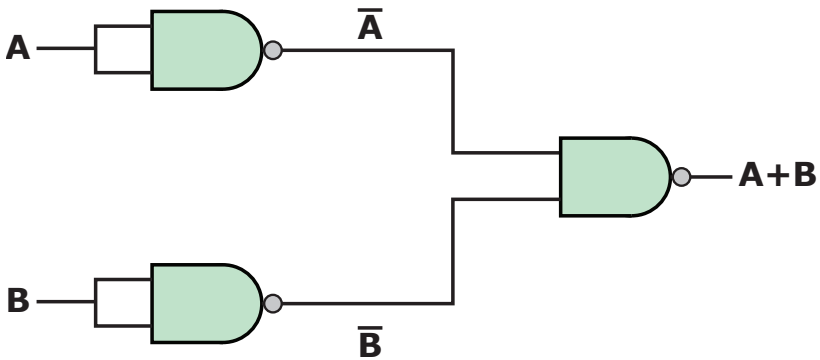
NAND and NOR

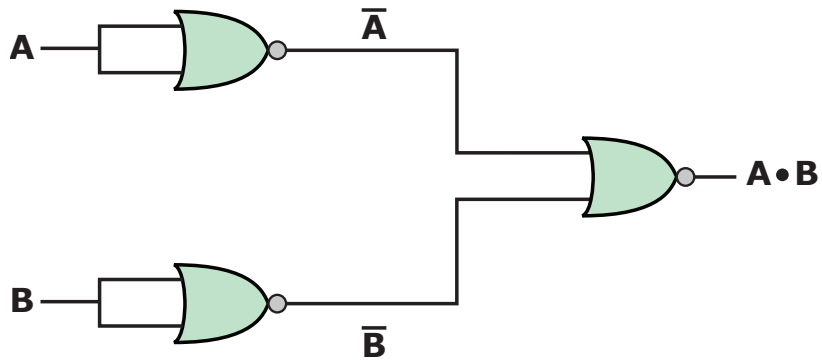
Can be used to implement every other basic operator.











- Boolean Algebra
- Gates and Circuits
- **Combinational Circuits**
- Minimizing Circuits
- Important Combinational Circuits
- Sequential Circuits

Combinational Circuits

- A circuit where the output is only a function of the current input.
- (As opposed to also be a function of the history of inputs)
- Notation: n binary inputs, m binary outputs
- Three schemes for definition:
  - Truth table: $n + m$ columns, 2^n rows, explicitly list the output for each input combination, with rows in a reasonable order.
  - Graphical: a drawing of the circuit with gates.
  - Boolean equations: each output expressed as a Boolean function of the inputs.
- We can relatively easily convert between each scheme.
- But there may be many different circuits/equations implementing the same function.

Small Example, Truth Table

A	B	C	$\boldsymbol{F}$
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

Small Example, to Boolean Function

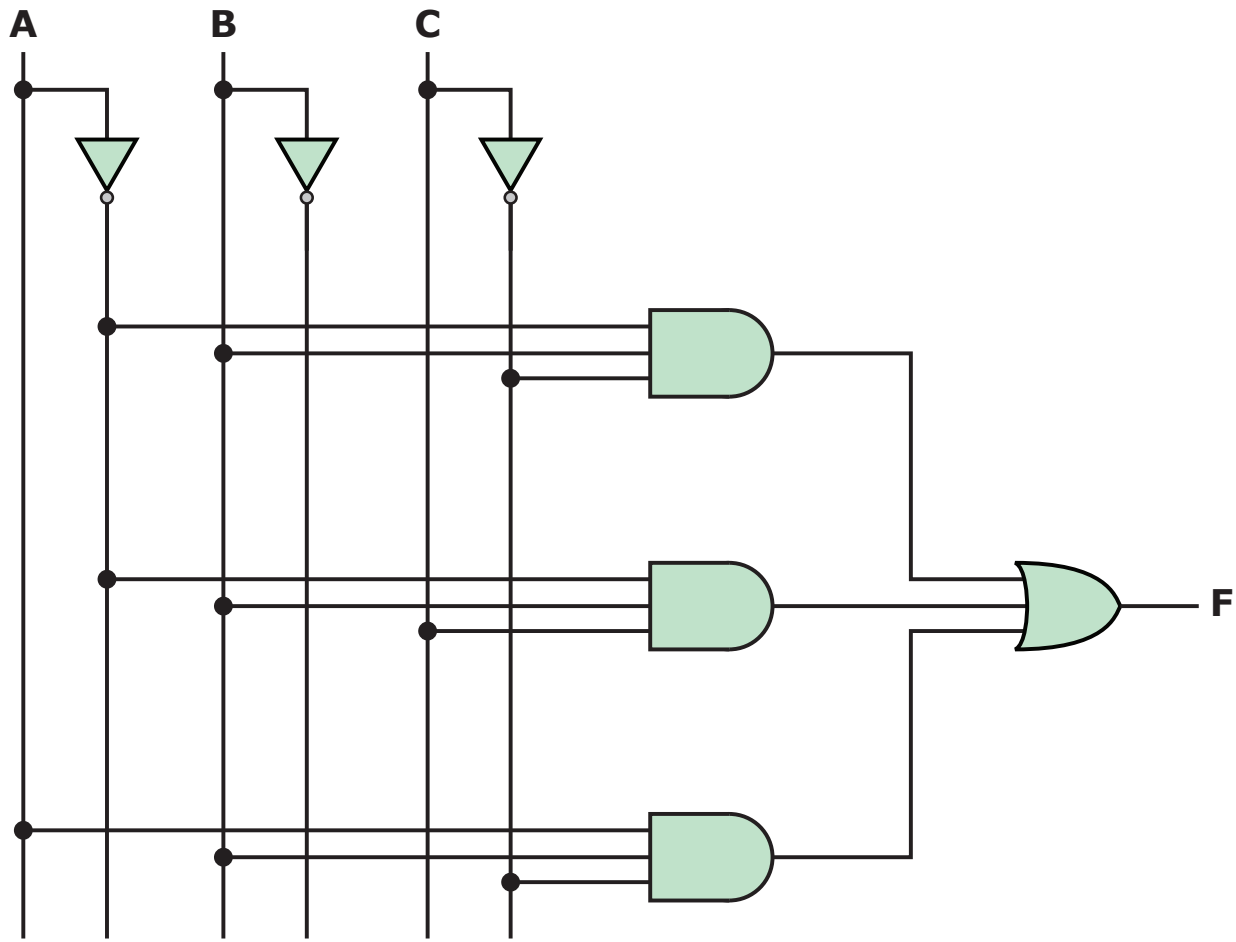
- We can rather easily transform the truth table into a boolean expression.
- It may be horribly large though.
- We can simply base it on the 1-rows: F is 1 when either
  - $\overline{A}\,B\,\overline{C}$ OR
  - $\overline{A}\,B\,C$ OR
  - $A\,B\,\overline{C}$ is 1.
- $F = \overline{A}\,B\,\overline{C} + \overline{A}\,B\,C + A\,B\,\overline{C}$
- This is called the sum of products (SOP) form.

A	B	C	F
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

Small Example, to Circuit

The expression can be converted directly to a circuit.

$F = \overline{A}\,B\,\overline{C} + \overline{A}\,B\,C + A\,B\,\overline{C}$



A	B	C	F
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

Small Example: Another Boolean Function

- We could also look at the positions where F is 0:
  - NOT $\overline{A}\,\overline{B}\,\overline{C}$ AND
  - NOT $\overline{A}\,\overline{B}\,C$ AND
  - NOT $A\,\overline{B}\,\overline{C}$ AND
  - NOT $A\,\overline{B}\,C$ AND
  - NOT $A\,B\,C$

A	B	C	$\boldsymbol{F}$
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

Small Example: Another Boolean Function

- We could also look at the positions where F is 0:
  - NOT $\overline{A}\,\overline{B}\,\overline{C}$ AND
  - NOT $\overline{A}\,\overline{B}\,C$ AND
  - NOT $A\,\overline{B}\,\overline{C}$ AND
  - NOT $A\,\overline{B}\,C$ AND
  - NOT $A\,B\,C$

- The output is 1 when all of those terms are 0:
  $F = \left(\overline{\overline{A}\,\overline{B}\,\overline{C}}\right) \cdot \left(\overline{\overline{A}\,\overline{B}\,C}\right) \cdot \left(\overline{A\,\overline{B}\,\overline{C}}\right) \cdot \left(\overline{A\,\overline{B}\,C}\right) \cdot \left(\overline{A\,B\,C}\right)$

A	B	C	$\boldsymbol{F}$
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

Small Example: Another Boolean Function

- We could also look at the positions where F is 0:
  - NOT $\overline{A}\,\overline{B}\,\overline{C}$ AND
  - NOT $\overline{A}\,\overline{B}\,C$ AND
  - NOT $A\,\overline{B}\,\overline{C}$ AND
  - NOT $A\,\overline{B}\,C$ AND
  - NOT $A\,B\,C$

A	B	C	F
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

- The output is 1 when all of those terms are 0:
$$F = \left(\overline{\overline{A}\,\overline{B}\,\overline{C}}\right) \cdot \left(\overline{\overline{A}\,\overline{B}\,C}\right) \cdot \left(\overline{A\,\overline{B}\,\overline{C}}\right) \cdot \left(\overline{A\,\overline{B}\,C}\right) \cdot \left(\overline{A\,B\,C}\right)$$

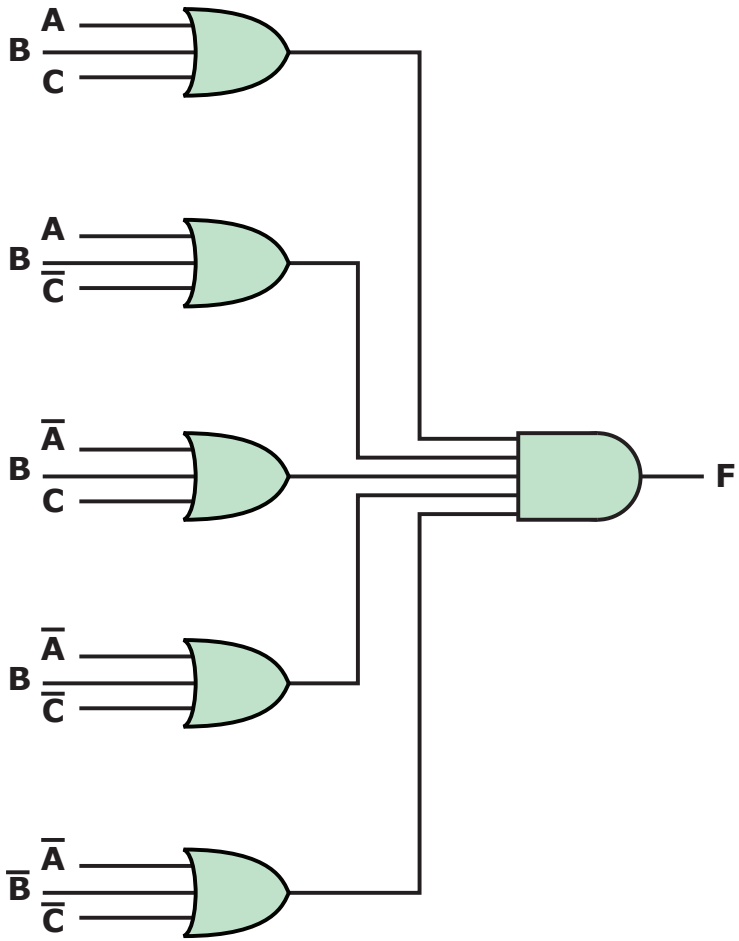
- Using DeMorgan's laws, and simplifying double negations:
$$F = (A + B + C) \cdot \left(A + B + \overline{C}\right) \cdot \left(\overline{A} + B + C\right) \cdot \left(\overline{A} + B + \overline{C}\right) \cdot \left(\overline{A} + \overline{B} + \overline{C}\right)$$

- This is the product of sums (POS) form.

Small Example, to Circuit

The POS expression can also be converted directly to a circuit.

$$F = (A + B + C) \cdot \left(A + B + \overline{C}\right) \cdot \left(\overline{A} + B + C\right) \cdot \left(\overline{A} + B + \overline{C}\right) \cdot \left(\overline{A} + \overline{B} + \overline{C}\right)$$



A	B	C	F
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**

- Boolean Algebra
- Gates and Circuits
- Combinational Circuits
- **Minimizing Circuits**
- Important Combinational Circuits
- Sequential Circuits

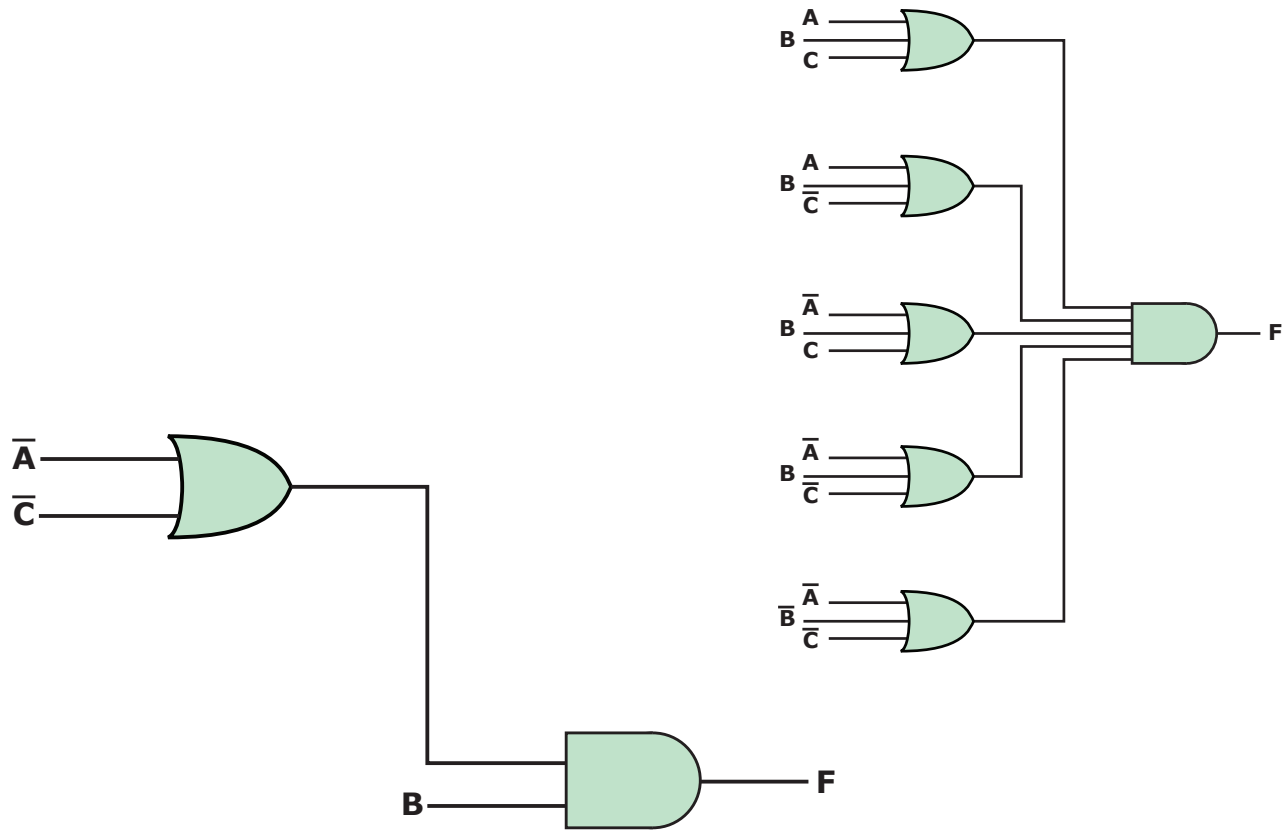
Implementation as a Circuit

- We have now seen two expressions/circuits for the same function.
- There may be many more equivalent circuits.
- The SOP and POS circuits are very systematic, but potentially very large.
- The SOP and POS circuits also used more than 2 inputs per gate.
- Perhaps we want to minimize the number of gates used.
- Perhaps we can use other gate types, e.g., NAND and NOR.
- Perhaps we want to minimize the use of certain gates.
- Perhaps we want to minimize the total gate delay, i.e., the depth.
- Not all circuits are realistic in a chip (too many wires).

Small Example, Simplified Implementation

$$F = B \cdot (\overline{A} + \overline{C})$$

A	B	C	F
0	0	0	**0**
0	0	1	**0**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**0**



How to Simplify Expressions?

- Direct algebraic simplification
  - Cumbersome and difficult.
- Karnaugh Maps
  - Graphical aid for a specific type of algebraic simplification.
  - Relatively easy
  - Though, rather difficult if more than 4 inputs.
- Quine-McCluskey Tables
  - An algorithmic approach.
  - Generalizes easily to more than 4 inputs.

Karnaugh Maps

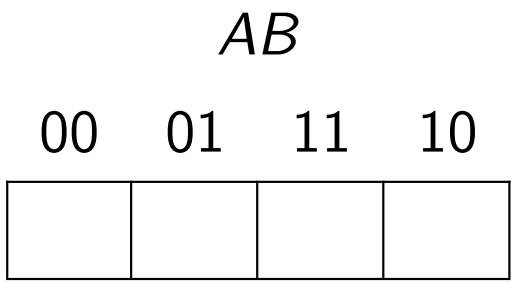
- We start with a SOP expression.
- Simplification idea: combine
  - extraction of common factors with
  - simplifying complement elements, $A + \overline{A} = 1$
- E.g., $F = A\,B + A\,\overline{B}$:
  - A is a common factor,
  - and extracting it gives an OR of complement elements: $A \cdot (B + \overline{B})$,
  - so $F = A$ is a simpler equivalent expression.
- A Karnaugh map is a visual aid to find common factors.

Karnaugh Maps

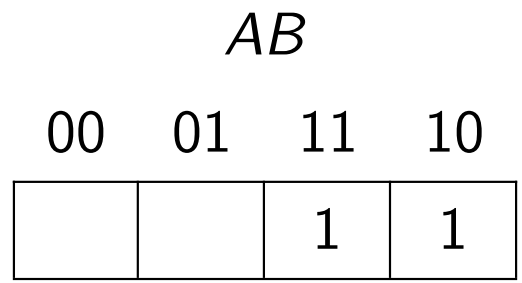
- We start with a SOP expression.
- Simplification idea: combine
  - extraction of common factors with
  - simplifying complement elements, $A + \overline{A} = 1$
- E.g., $F = A\,B + A\,\overline{B}$:
  - A is a common factor,
  - and extracting it gives an OR of complement elements: $A \cdot (B + \overline{B})$,
  - so $F = A$ is a simpler equivalent expression.
- A Karnaugh map is a visual aid to find common factors.
- A truth table for two variables, A and B has 4 rows.
- Make a grid with a cell for each row,
- but place them so adjacent cells only differs in 1 variable.
- E.g., the cells for $A\,B$ and $A\,\overline{B}$ should be adjacent.
- Actually, we need a torus for this (a grid with wrap-around).
- We will have different grids for 2, 3, and 4 variables.

Karnaugh Maps

- 2 input variables: A, B



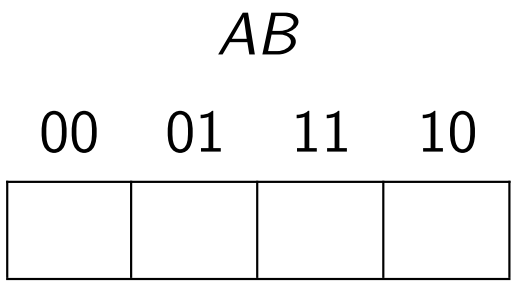
Empty map.



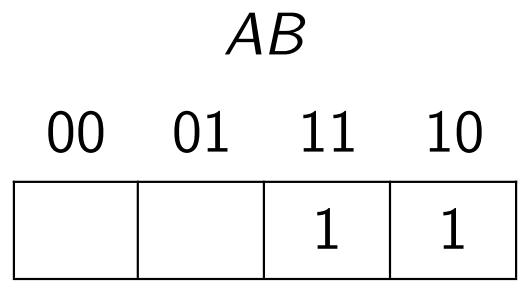
Map for $A\,B + A\,\overline{B}$

Karnaugh Maps

- 2 input variables: A, B

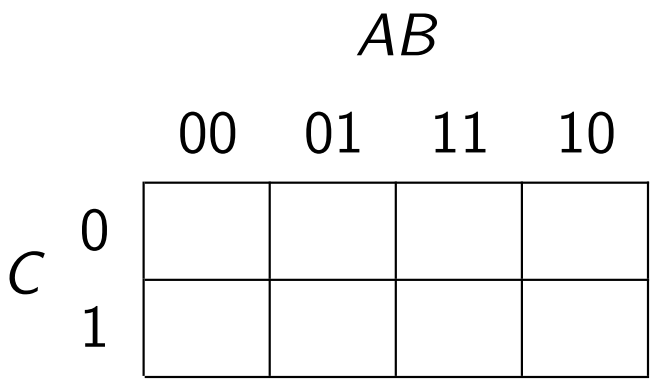


Empty map.

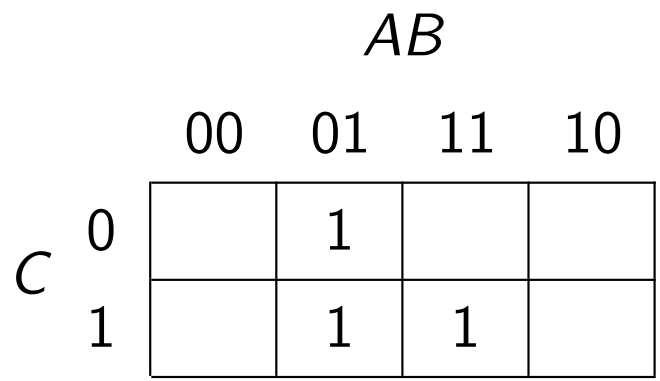


Map for $A\,B + A\,\overline{B}$

- 3 input variables: A, B, C

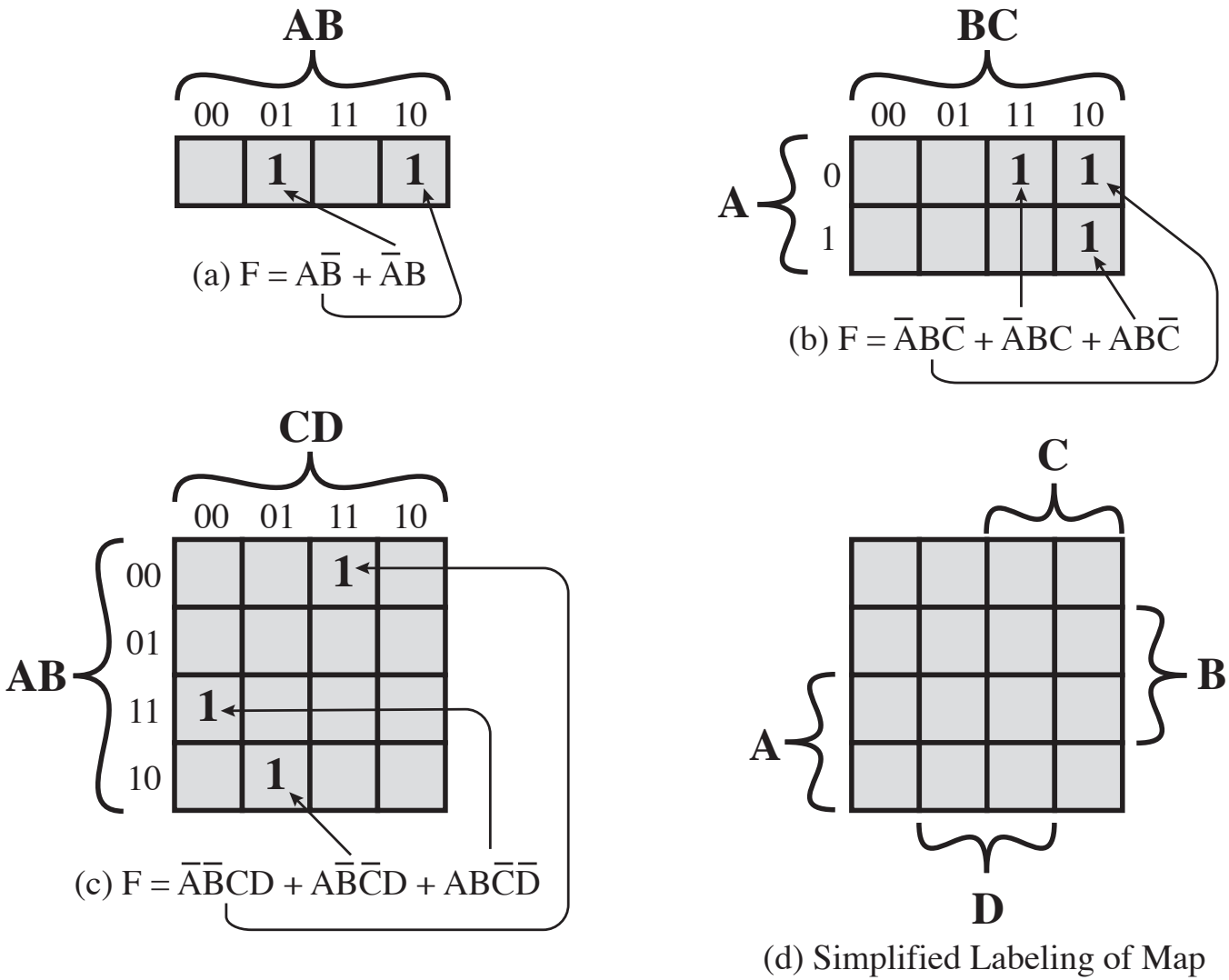


Empty map.



Map for $\overline{A}\,B\,\overline{C} + \overline{A}\,B\,C + A\,B\,C$

Karnaugh Maps



How does it help?

- Two adjacent cells differ in 1 variable,
- so extract and eliminate.
- $F = A \cdot (B + \overline{B}) = A$.
- Actually, we will write the expression directly from the table.

AB	00	01	11	10
			1	1

Map for $F = A\,B + A\,\overline{B}$

		AB			
		00	01	11	10
C	0		1		
	1		1	1	

Map for
$G = \overline{A}\,B\,\overline{C} + \overline{A}\,B\,C + A\,B\,C$

How does it help?

- Two adjacent cells differ in 1 variable,
- so extract and eliminate.
- $F = A \cdot (B + \overline{B}) = A$.
- Actually, we will write the expression directly from the table.

AB	00	01	11	10
			1	1

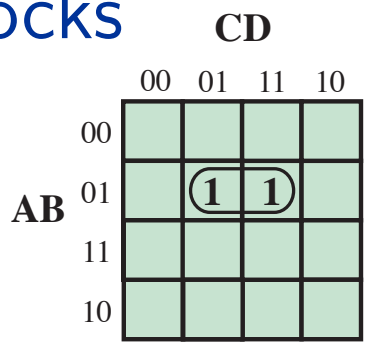
Map for $F = A\,B + A\,\overline{B}$

- To make an equivalent expression, we must cover all 1-cells, and none of the 0-cells
- It doesn't matter if a 1-cell is covered by multiple terms.
- For G, we have a vertical block and a horizontal block.
- so $G = \overline{A}\,B + B\,C$

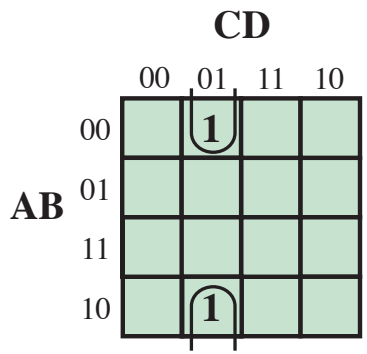
C \ AB	00	01	11	10
0		1		
1		1	1	

Map for
$G = \overline{A}\,B\,\overline{C} + \overline{A}\,B\,C + A\,B\,C$

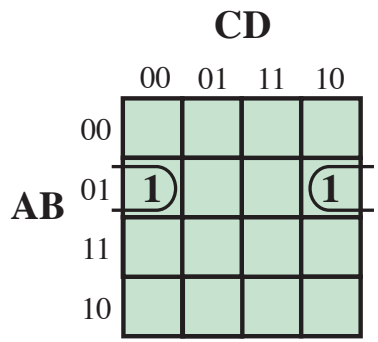
Larger Blocks



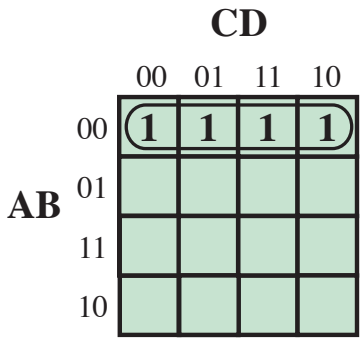
(a) $\bar{A}BD$



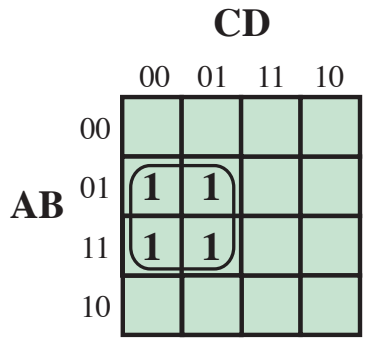
(b) $\bar{B}\bar{C}D$



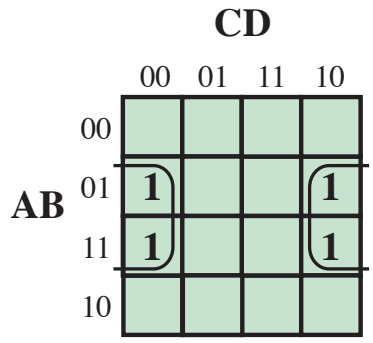
(c) $\bar{A}B\bar{D}$



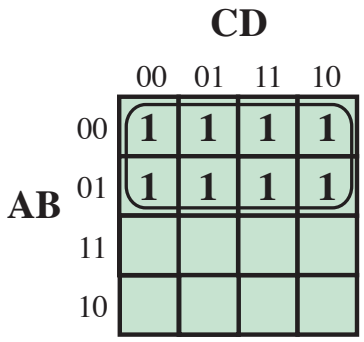
(d) $\bar{A}\bar{B}$



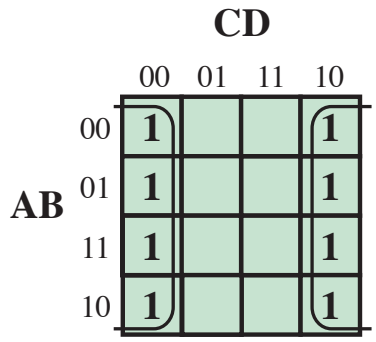
(e) $B\bar{C}$



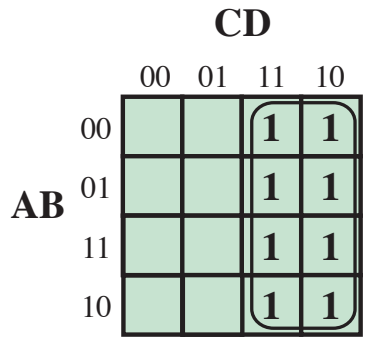
(f) $B\bar{D}$



(g) $\bar{A}$



(h) $\bar{D}$

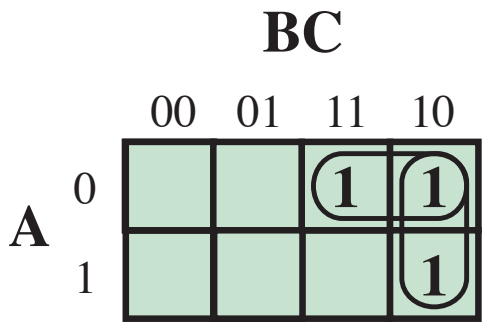


(i) C

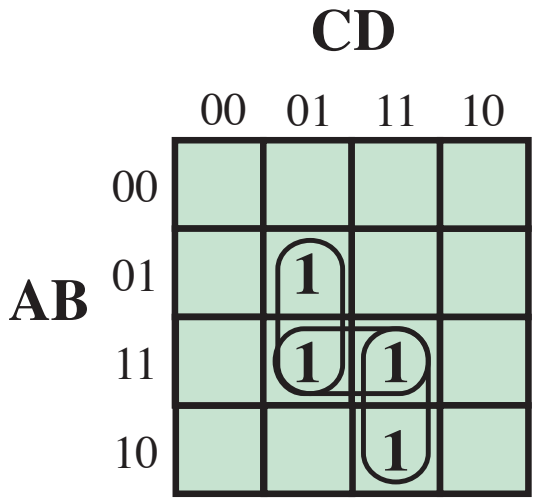
Karnaugh Map Algorithm

1. Make the map, filling in 1's similarly to how you fill a truth table.
2. Mark the largest rectangles of 1, 2, 4, or 8 1-cells.
   - All 1-cells must be covered.
   - No 0-cells must be covered.
   - A 1-cell may be in several blocks.
3. Write down the SOP expression where each block is a term.

Examples



(a) $F = \bar{A}B + B\bar{C}$



(b) $F = B\bar{C}D + ACD$

We just need to cover all 1-cells, so with overlaps, just take as few and as large as possible.

Example: Decimal Incrementer

- Sometimes we know that only some input combinations will be used.
- So for we don't care about what the output is.
- This is marked with d.
- Depending on what is convenient, we can let each be 0 or 1.

i	a_3	a_2	a_1	a_0	$i+1$	f_3	f_2	f_1	f_0
0	0	0	0	0	1	0	0	0	1
1	0	0	0	1	2	0	0	1	0
2	0	0	1	0	3	0	0	1	1
3	0	0	1	1	4	0	1	0	0
4	0	1	0	0	5	0	1	0	1
5	0	1	0	1	6	0	1	1	0
6	0	1	1	0	7	0	1	1	1
7	0	1	1	1	8	1	0	0	0
8	1	0	0	0	9	1	0	0	1
9	1	0	0	1	0	0	0	0	0
—	1	0	1	0		d	d	d	d
—	1	0	1	1		d	d	d	d
—	1	1	0	0		d	d	d	d
—	1	1	0	1		d	d	d	d
—	1	1	1	0		d	d	d	d
—	1	1	1	1		d	d	d	d

Karnaugh Maps: Decimal Incrementer

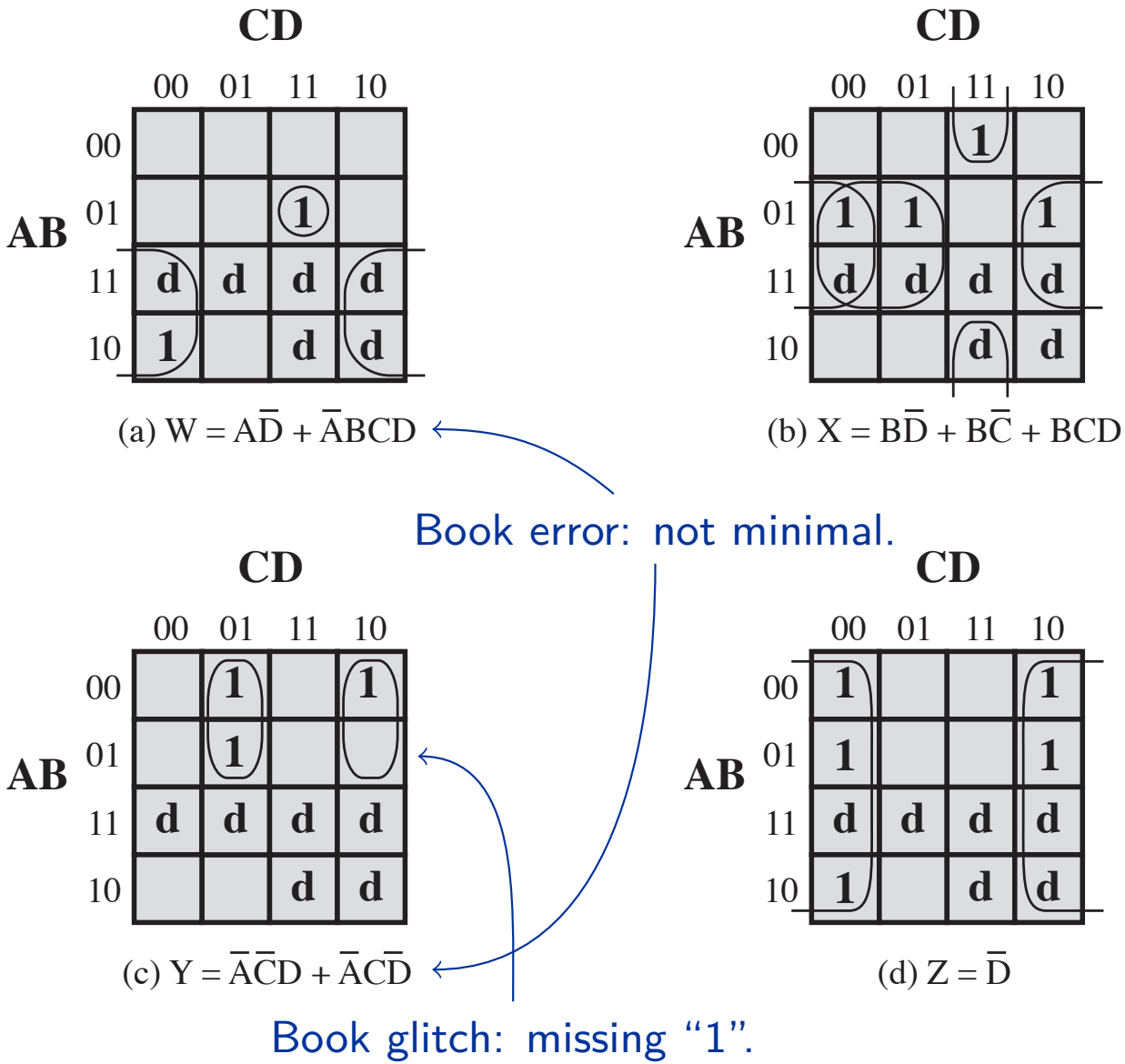


Figure 11.10 Karnaugh Maps for the Incrementer

The Quine-McCluskey Method

- Karnaugh maps become very complicated for more than 4 input variables: drawing 3D boxes on a 4D torus.
- But we can use the same ideas in a different manner, e.g., in the Quine-McCluskey method.
- As before, we start from a SOP expression and combine
  - extraction of common factors with
  - simplifying complement elements, $A + \overline{A} = 1$
- Systematically enumerate all possible ways of doing that combined operation.
- E.g., for $F = ABCD + AB\overline{C}D + AB\overline{C}\,\overline{D}$
  We can create both
  - $ABD(C + \overline{C}) = ABD$
    and that term will now cover $ABCD$ and $AB\overline{C}D$.
  - $AB\overline{C}(D + \overline{D}) = AB\overline{C}$
    and that term will now cover $AB\overline{C}D$ and $AB\overline{C}\,\overline{D}$.

The Quine-McCluskey Method

- We don't need to check all pairs of terms:
  - We just want to simplify 1 variable at a time, so the terms must have the same size.
  - We want to find pairs that just differ in 1 variable, so one term must have exactly 1 more negated variable than the other.

The Quine-McCluskey Method

- We don't need to check all pairs of terms:
  - We just want to simplify 1 variable at a time, so the terms must have the same size.
  - We want to find pairs that just differ in 1 variable, so one term must have exactly 1 more negated variable than the other.
- Idea for the first phase:
  - For an expression over k variables, make a $k \times k$ table.
  - The first row contains terms over k variables (the input).
  - The second row contains terms over $k - 1$ variables (to be generated).
  - ...

The Quine-McCluskey Method

- We don't need to check all pairs of terms:
  - We just want to simplify 1 variable at a time, so the terms must have the same size.
  - We want to find pairs that just differ in 1 variable, so one term must have exactly 1 more negated variable than the other.
- Idea for the first phase:
  - For an expression over k variables, make a $k \times k$ table.
  - The first row contains terms over k variables (the input).
  - The second row contains terms over $k-1$ variables (to be generated).
  - ...
  - Each column contains terms with the same number of negated variables.
  - Candidate pairs are then in the same row, in neighbouring columns.
  - Mark terms we have covered by newly generated terms.

Example: $F = ABCD + AB\overline{C}D + AB\overline{C}\,\overline{D} + A\overline{B}CD + \overline{A}BCD + \overline{A}BC\overline{D} + \overline{A}B\overline{C}D + \overline{A}\,\overline{B}\,\overline{C}D$

Example

$ABCD$ $AB\overline{C}D$ $AB\overline{C}\,\overline{D}$ $\overline{A}\,\overline{B}\,\overline{C}D$

$A\overline{B}CD$ $\overline{A}BC\overline{D}$

$\overline{A}BCD$ $\overline{A}B\overline{C}D$

Example

$ABCD\checkmark$ —?— $AB\overline{C}D\checkmark$ $AB\overline{C}\,\overline{D}$ $\overline{A}\,\overline{B}\,\overline{C}D$

$A\overline{B}CD$ $\overline{A}BC\overline{D}$

$\overline{A}BCD$ $\overline{A}B\overline{C}D$

ABD

Example

$ABCD\checkmark$? $AB\overline{C}D\checkmark$ $AB\overline{C}\,\overline{D}$ $\overline{A}\,\overline{B}\,\overline{C}D$

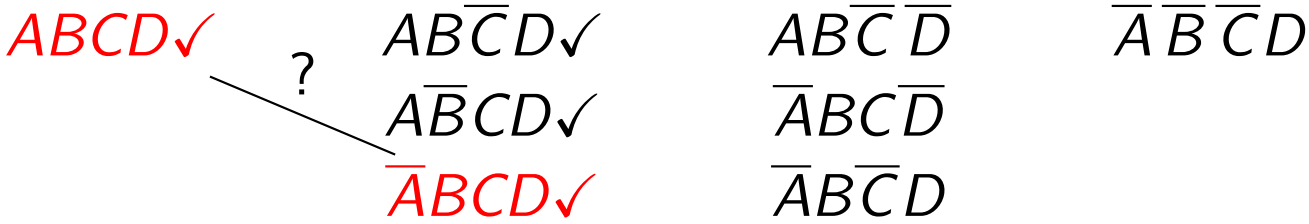
$A\overline{B}CD\checkmark$ $\overline{A}BC\overline{D}$

$\overline{A}BCD$ $\overline{A}B\overline{C}D$

ABD

ACD

Example



ABD

ACD

BCD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$? $AB\overline{C}\,\overline{D}\checkmark$

$A\overline{B}CD\checkmark$

$\overline{A}BCD\checkmark$

$\overline{A}BC\overline{D}$

$\overline{A}B\overline{C}D$

$\overline{A}\,\overline{B}\,\overline{C}D$

ABD

ACD

BCD

$AB\overline{C}$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$? $AB\overline{C}\,\overline{D}\checkmark$

$A\overline{B}CD\checkmark$ $\overline{A}BC\overline{D}$

$\overline{A}BC\overline{D}\checkmark$ $\overline{A}B\overline{C}D$

$\overline{A}\,\overline{B}\,\overline{C}D$

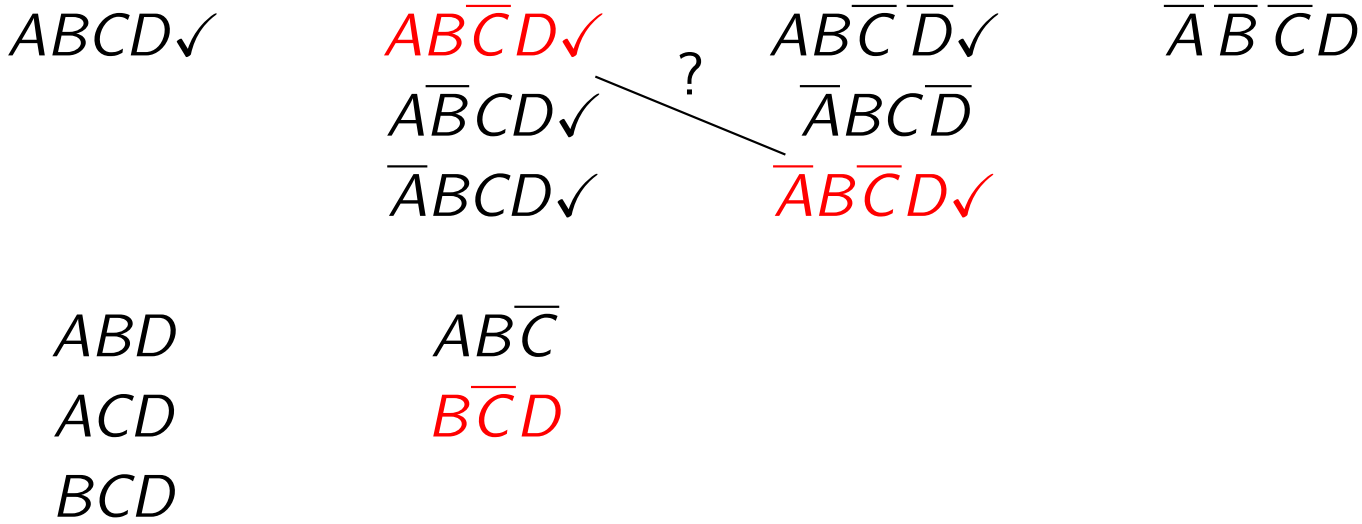
ABD

ACD

BCD

$AB\overline{C}$

Example


$ABCD\checkmark$
$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$
?
$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}$
$\overline{A}B\overline{C}D\checkmark$
$\overline{A}\,\overline{B}\,\overline{C}D$
ABD
ACD
BCD
$AB\overline{C}$
$B\overline{C}D$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$? $A\overline{B}CD\checkmark$

$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$

$\overline{A}BC\overline{D}$

$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D$

ABD

ACD

BCD

$AB\overline{C}$

$B\overline{C}D$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$

$A\overline{B}CD\checkmark$

$\overline{A}BCD\checkmark$

?

$AB\overline{C}\,\overline{D}\checkmark$

$\overline{A}BC\overline{D}$

$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D$

ABD

ACD

BCD

$AB\overline{C}$

$B\overline{C}D$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$

$A\overline{B}CD\checkmark$

$\overline{A}BCD\checkmark$

?

$AB\overline{C}\,\overline{D}\checkmark$

$\overline{A}BC\overline{D}$

$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D$

ABD

ACD

BCD

$AB\overline{C}$

$B\overline{C}D$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$

$A\overline{B}CD\checkmark$

$\overline{A}BCD\checkmark$

?

$AB\overline{C}\,\overline{D}\checkmark$

$\overline{A}BC\overline{D}$

$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D$

ABD

ACD

BCD

$AB\overline{C}$

$B\overline{C}D$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

?

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D$

ABD
ACD
BCD

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

?

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D$

ABD
ACD
BCD

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark \overset{?}{\text{———}} \overline{A}\,\overline{B}\,\overline{C}D$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

ABD
ACD
BCD

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$?$ — $\overline{A}\,\overline{B}\,\overline{C}D$

ABD
ACD
BCD

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

?

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

ABD
ACD
BCD

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD$

$\overline{A}\,\overline{C}D$

Example

$ABCD\checkmark$	$AB\overline{C}D\checkmark$	$AB\overline{C}\,\overline{D}\checkmark$	$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$
	$A\overline{B}CD\checkmark$	$\overline{A}BC\overline{D}\checkmark$	
	$\overline{A}BCD\checkmark$	$\overline{A}B\overline{C}D\checkmark$	

	?		
ABD	——	$AB\overline{C}$	$\overline{A}\,C\,D$
ACD		$B\overline{C}D$	
BCD		$\overline{A}BC$	
		$\overline{A}BD$	

Example

$ABCD\checkmark$	$AB\overline{C}D\checkmark$	$AB\overline{C}\,\overline{D}\checkmark$	$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$
	$A\overline{B}CD\checkmark$	$\overline{A}BC\overline{D}\checkmark$	
	$\overline{A}BCD\checkmark$	$\overline{A}B\overline{C}D\checkmark$	

ABD	?	$AB\overline{C}$	$\overline{A}\,\overline{C}D$
ACD		$B\overline{C}D$	
BCD		$\overline{A}BC$	
		$\overline{A}BD$	

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

ABD
ACD
BCD

?

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD$

$\overline{A}\,\overline{C}D$

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
BCD

?

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$

$\overline{A}\,\overline{C}D$

$\overline{A}BD\checkmark$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
BCD

?

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
BCD

?

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
BCD

?

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
BCD

?

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
BCD

?

$AB\overline{C}$
$B\overline{C}D$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
$BCD\checkmark$

?

$AB\overline{C}$
$B\overline{C}D\checkmark$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
$BCD\checkmark$ — ? — $AB\overline{C}$
$B\overline{C}D\checkmark$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
$BCD\checkmark$? $AB\overline{C}$
$B\overline{C}D\checkmark$
$\overline{A}BC$
$\overline{A}BD\checkmark$

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
$BCD\checkmark$

$AB\overline{C}$ —?— $\overline{A}\,\overline{C}D$
$B\overline{C}D\checkmark$
$\overline{A}BC$
$\overline{A}BD\checkmark$

BD

Example

$ABCD\checkmark$ $AB\overline{C}D\checkmark$ $AB\overline{C}\,\overline{D}\checkmark$ $\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$A\overline{B}CD\checkmark$ $\overline{A}BC\overline{D}\checkmark$

$\overline{A}BCD\checkmark$ $\overline{A}B\overline{C}D\checkmark$

$ABD\checkmark$ $AB\overline{C}$? $\overline{A}\,\overline{C}D$

ACD $B\overline{C}D\checkmark$

$BCD\checkmark$ $\overline{A}BC$

$\overline{A}BD\checkmark$

BD

Example

$ABCD\checkmark$

$AB\overline{C}D\checkmark$
$A\overline{B}CD\checkmark$
$\overline{A}BCD\checkmark$

$AB\overline{C}\,\overline{D}\checkmark$
$\overline{A}BC\overline{D}\checkmark$
$\overline{A}B\overline{C}D\checkmark$

$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$ABD\checkmark$
ACD
$BCD\checkmark$

$AB\overline{C}$
$B\overline{C}D\checkmark$
$\overline{A}BC$
$\overline{A}BD\checkmark$

?

$\overline{A}\,\overline{C}D$

BD

Example

$ABCD\checkmark$ $AB\overline{C}D\checkmark$ $AB\overline{C}\,\overline{D}\checkmark$ $\overline{A}\,\overline{B}\,\overline{C}D\checkmark$

$A\overline{B}CD\checkmark$ $\overline{A}BC\overline{D}\checkmark$

$\overline{A}BC\overline{D}\checkmark$ $\overline{A}B\overline{C}D\checkmark$

$ABD\checkmark$ $AB\overline{C}$ $\overline{A}\,\overline{C}D$

ACD $B\overline{C}D\checkmark$?

$BCD\checkmark$ $\overline{A}BC$

$\overline{A}BD\checkmark$

BD

Example

$ABCD\checkmark$	$AB\overline{C}D\checkmark$	$AB\overline{C}\,\overline{D}\checkmark$	$\overline{A}\,\overline{B}\,\overline{C}D\checkmark$
	$A\overline{B}CD\checkmark$	$\overline{A}BC\overline{D}\checkmark$	
	$\overline{A}BCD\checkmark$	$\overline{A}B\overline{C}D\checkmark$	
$ABD\checkmark$	$AB\overline{C}$	$\overline{A}\,\overline{C}D$	
ACD	$B\overline{C}D\checkmark$		
$BCD\checkmark$	$\overline{A}BC$		
	$\overline{A}BD\checkmark$		
BD			

- Now we can take all non-covered terms as a reduced form.
- $F = ACD + AB\overline{C} + \overline{A}BC + \overline{A}\,\overline{C}D + BD$
- But perhaps we can improve on this: we don't need to cover every term, only the original ones.

The Quine-McCluskey Method, Phase 2

1. Make a new table: columns are the input terms, rows are the reduced terms.
2. Mark each cell if the reduced term covers the input term.

	$ABCD$	$AB\overline{C}D$	$AB\overline{C}\,\overline{D}$	$A\overline{B}CD$	$\overline{A}BCD$	$\overline{A}BC\overline{D}$	$\overline{A}B\overline{C}D$	$\overline{A}\,\overline{B}\,\overline{C}D$
ACD	○			○				
$AB\overline{C}$		○	○					
$\overline{A}BC$					○	○		
$\overline{A}\,\overline{C}D$							○	○
BD	○	○			○		○	

The Quine-McCluskey Method, Phase 2

1. Make a new table: columns are the input terms, rows are the reduced terms.
2. Mark each cell if the reduced term covers the input term.
3. Select the marked cells which are alone in their column, indicating we need those reduced terms.

	$ABCD$	$AB\overline{C}D$	$AB\overline{C}\,\overline{D}$	$A\overline{B}CD$	$\overline{A}BCD$	$\overline{A}BC\overline{D}$	$\overline{A}B\overline{C}D$	$\overline{A}\,\overline{B}\,\overline{C}D$
ACD	○			×				
$AB\overline{C}$		○	×					
$\overline{A}BC$					○	×		
$\overline{A}\,\overline{C}D$							○	×
BD	○	○			○		○	

The Quine-McCluskey Method, Phase 2

1. Make a new table: columns are the input terms, rows are the reduced terms.
2. Mark each cell if the reduced term covers the input term.
3. Select the marked cells which are alone in their column, indicating we need those reduced terms.
4. But those terms may cover other columns, so select the whole row.

	$ABCD$	$AB\overline{C}D$	$AB\overline{C}\,\overline{D}$	$A\overline{B}CD$	$\overline{A}BCD$	$\overline{A}BC\overline{D}$	$\overline{A}B\overline{C}D$	$\overline{A}\,\overline{B}\,\overline{C}D$
ACD	□			×				
$AB\overline{C}$		□	×					
$\overline{A}BC$					□	×		
$\overline{A}\,\overline{C}D$							□	×
BD	○	○			○		○	

The Quine-McCluskey Method, Phase 2

1. Make a new table: columns are the input terms, rows are the reduced terms.
2. Mark each cell if the reduced term covers the input term.
3. Select the marked cells which are alone in their column, indicating we need those reduced terms.
4. But those terms may cover other columns, so select the whole row.
5. Select further rows (i.e., terms) until all columns are covered.

	$ABCD$	$AB\overline{C}D$	$AB\overline{C}\,\overline{D}$	$A\overline{B}CD$	$\overline{A}BCD$	$\overline{A}BC\overline{D}$	$\overline{A}B\overline{C}D$	$\overline{A}\,\overline{B}\,\overline{C}D$
ACD	□			×				
$AB\overline{C}$		□	×					
$\overline{A}BC$					□	×		
$\overline{A}\,\overline{C}D$							□	×
BD	○	○			○		○	

Example

- Input:
  $F = ABCD + AB\overline{C}D + AB\overline{C}\,\overline{D} + A\overline{B}CD$
  $+\overline{A}BCD + \overline{A}BC\overline{D} + \overline{A}B\overline{C}D + \overline{A}\,\overline{B}\,\overline{C}D$
- Reduced after phase 1:
  $F = ACD + AB\overline{C} + \overline{A}BC + \overline{A}\,\overline{C}D + BD$
- Reduced after phase 2:
  $F = ACD + AB\overline{C} + \overline{A}BC + \overline{A}\,\overline{C}D$

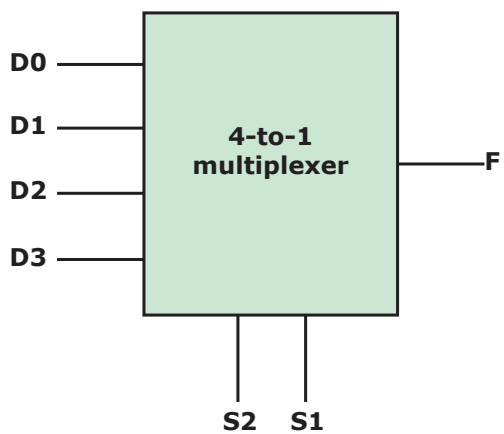
- Boolean Algebra
- Gates and Circuits
- Combinational Circuits
- Minimizing Circuits
- Important Combinational Circuits
- Sequential Circuits

Multiplexer

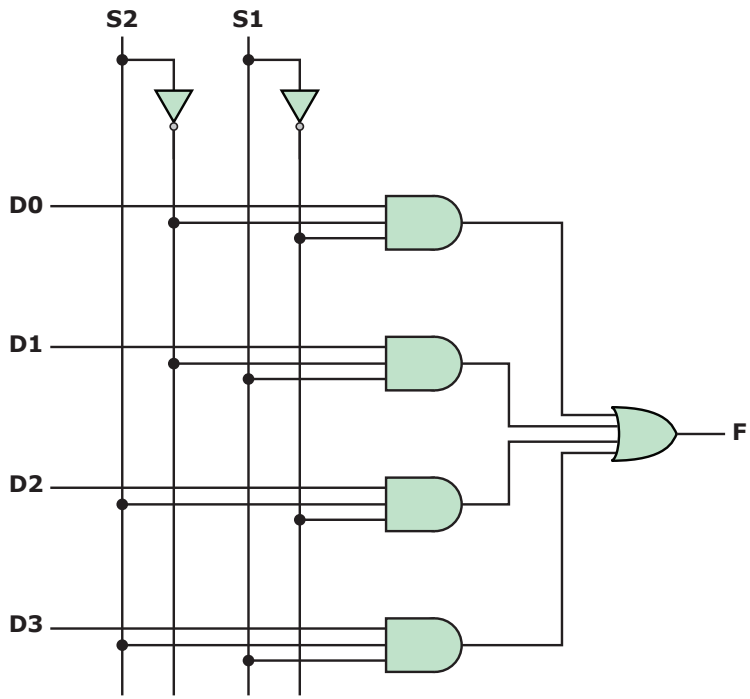
- Many inputs, one output, select which by control inputs.
- Useful for routing of signals/data.
- Abbreviated truth table:

S_1	S_2	F
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

- Abstract representation:

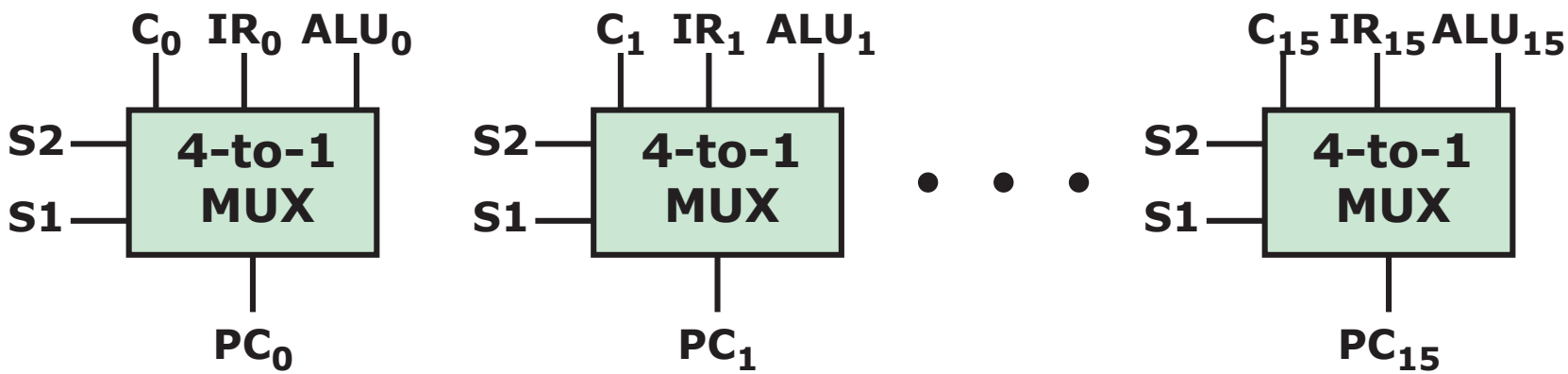


Implementation:



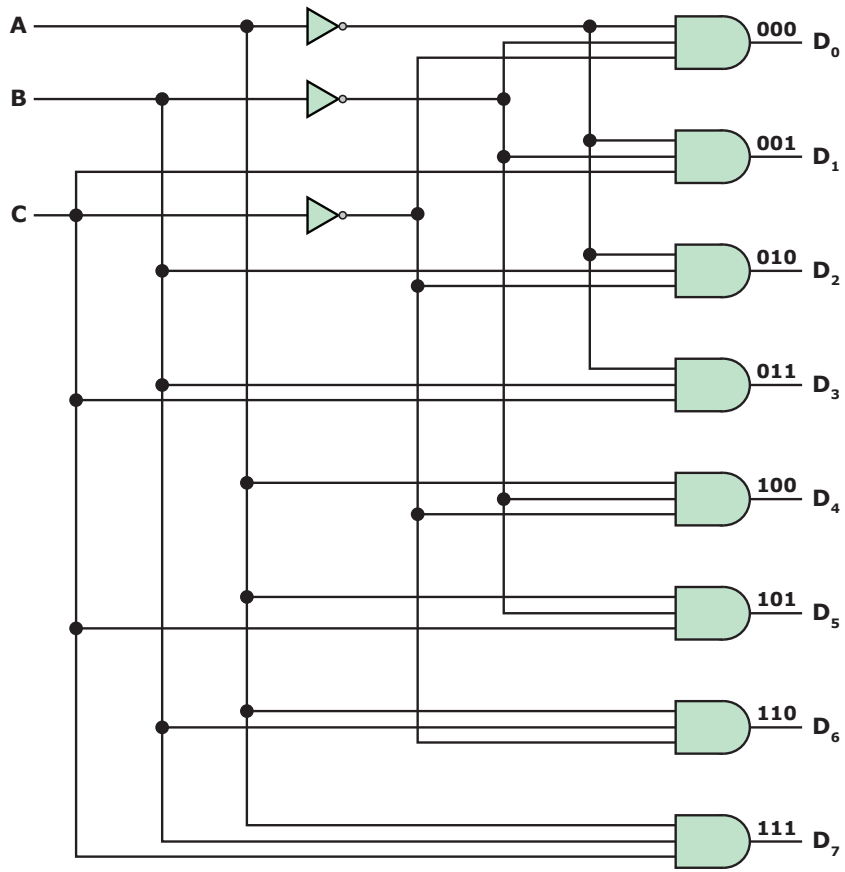
Multiplexer, Example

- What is the new value of the program counter?
  - Just the next instruction (i.e., $\mathrm{PC} + 1$)
  - A value in the current instruction (an unconditional jump).
  - Either of the above, depending on some ALU flag (conditional jump).
  - ...
- We need multiple multiplexers, one for each bit.



Decoder

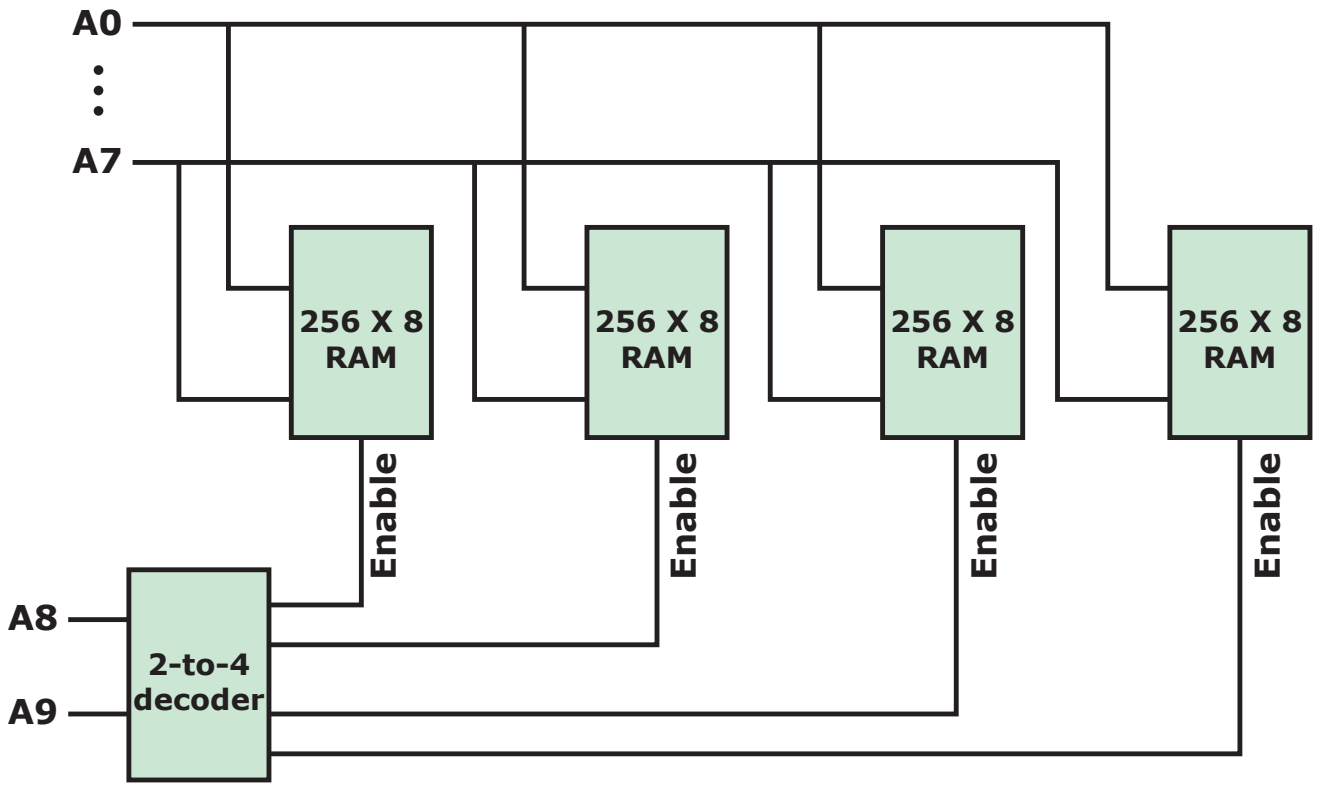
- Set exactly one of many output lines to 1, depending on input lines.
- Generally: n input lines and 2^n output lines.



Decoder, Example

RAM:

- Input: (part of) an address to read/write.
- Output: an "enabled" line to each RAM chip.

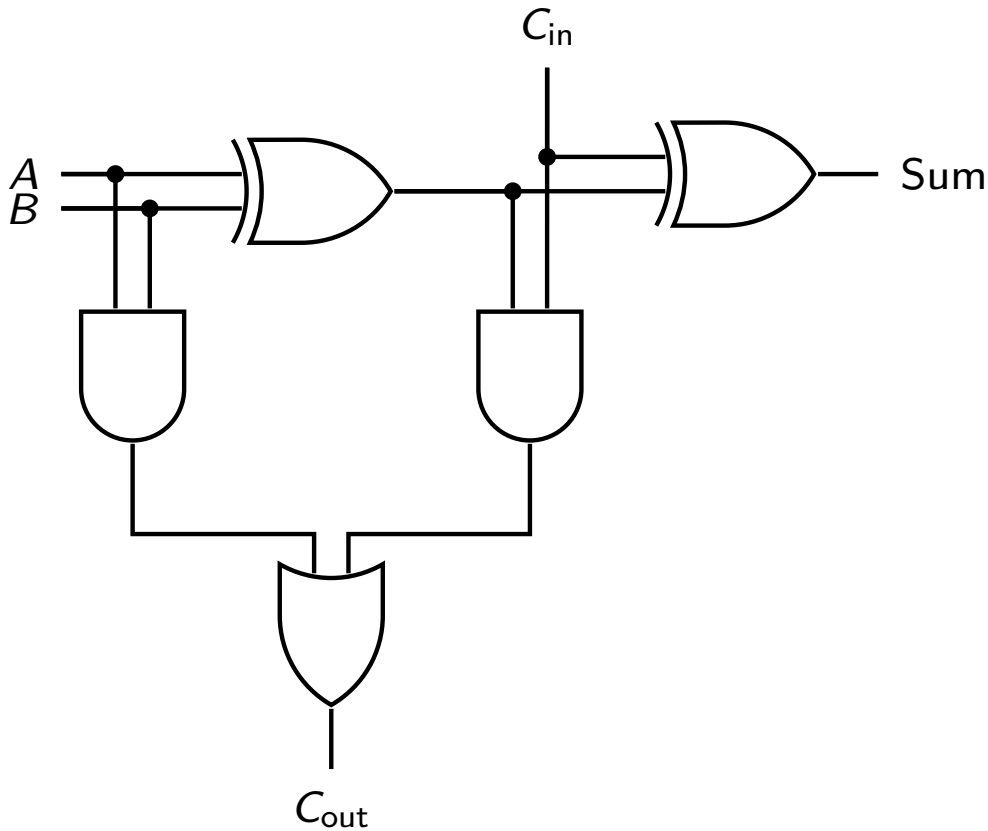


Adder

1-bit addition with carry out: half-adder

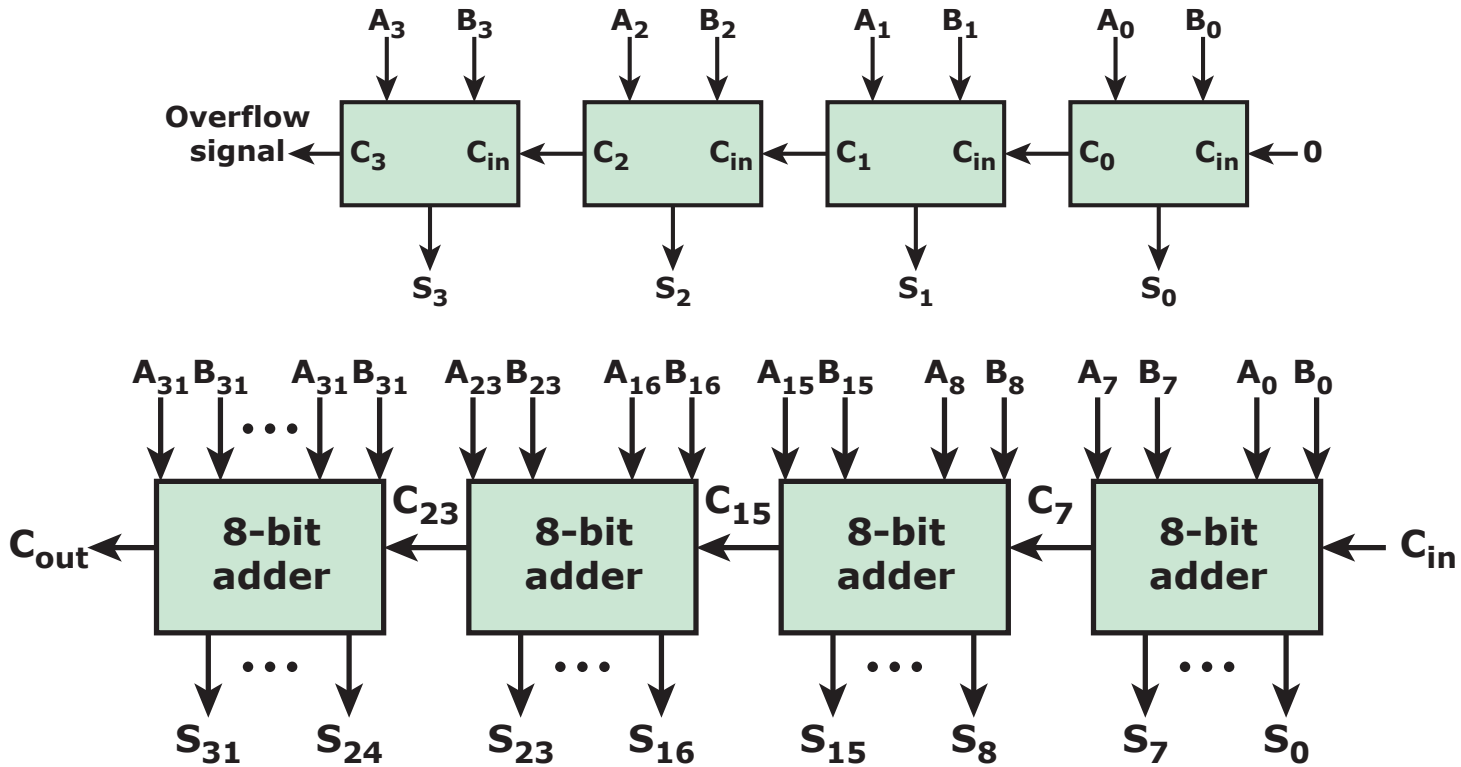
1-bit addition with carry in and out: full-adder

C_{in}	A	B	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



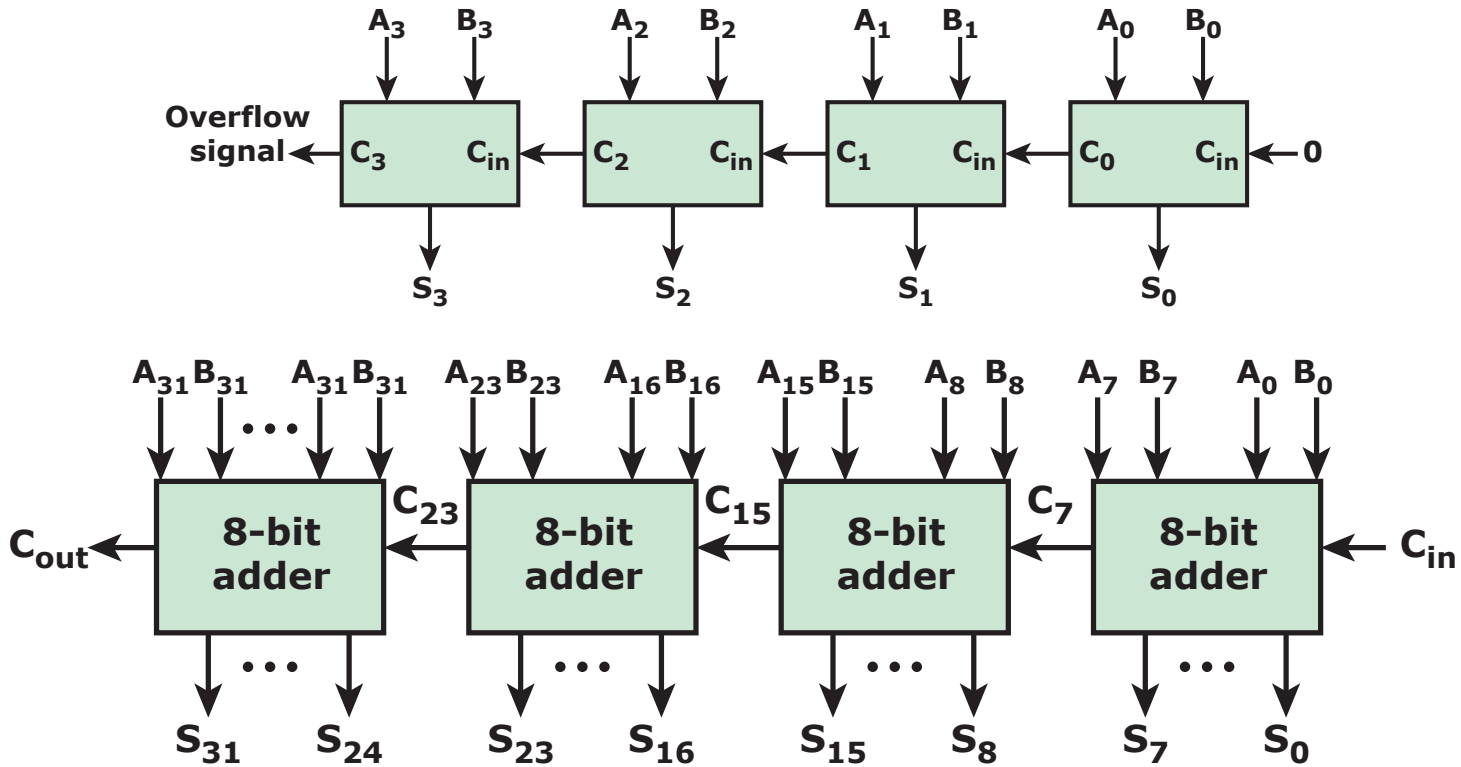
Multi-Bit Adder

- "Overflow signal" should here be called "carry out".
- For unsigned addition it is indeed the overflow signal,
- but for twos complement the rule is different.



Multi-Bit Adder

- "Overflow signal" should here be called "carry out".
- For unsigned addition it is indeed the overflow signal,
- but for twos complement the rule is different.



- This design is called a ripple-carry adder.
- What is the gate delay for the MSB of the result?
- This is unacceptable for large adders.

Faster Addition: Carry-Lookahead Adders

For a 1-bit addition $A_i + B_i$:

- When will we definitely generate a carry?: $G_i = A_i B_i$
- If we get a carry in, when will be produce a carry out? (propagate): $P_i = A_i + B_i$

so $C_i = G_i + P_i C_{i-1}$:

- either we generate a carry, or we pass one through from the previous position.

Carry-Lookahead Adders

We can expand those expressions: $C_i = G_i + P_i C_{i-1}$

- $C_0 = A_0 B_0$
- $C_1 = A_1 B_1 + (A_1 + B_1) \cdot A_0 B_0 = A_1 B_1 + A_1 A_0 B_0 + B_1 A_0 B_0$
- $C_2 = \cdots = A_2 B_2$
  $+A_2 A_1 B_2 + A_2 A_1 A_0 B_0 + A_2 B_1 A_0 B_0$
  $+B_2 A_1 B_2 + B_2 A_1 A_0 B_0 + B_2 B_1 A_0 B_0$

- Each carry can be expressed as a SOP of the input numbers.
- But it gets increasingly complex:
  - An n-bit adder requires $2^n - 1$ AND gates, and an OR gate with $2^n - 1$ inputs.
  - This is normally done up to at most 8 bit.
  - Larger adders are then built of 8-bit adders.
- Trade-off: gate delay vs. fan-in (see e.g., Kogge-Stone adder)

- Boolean Algebra
- Gates and Circuits
- Combinational Circuits
- Minimizing Circuits
- Important Combinational Circuits
- Sequential Circuits

Sequential Circuits

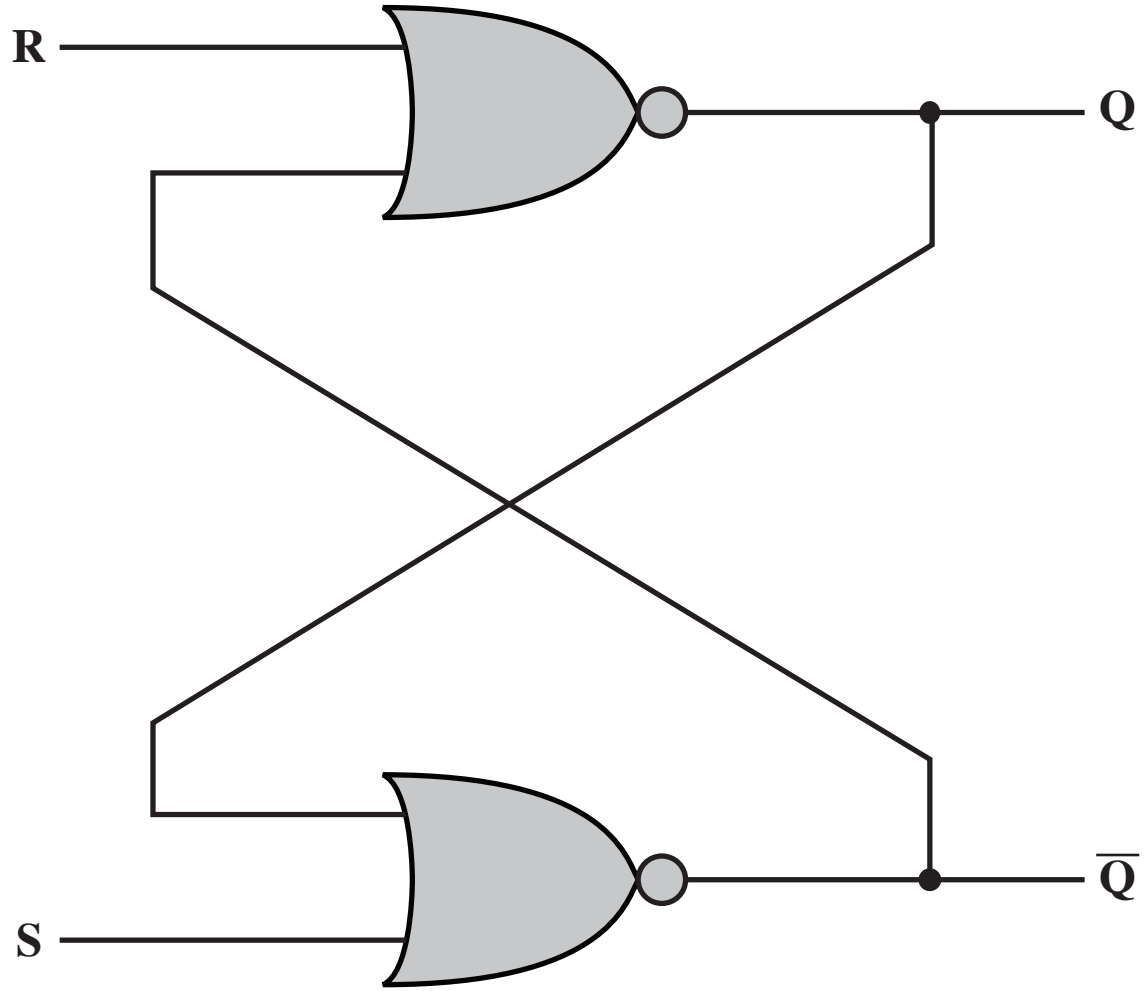
- Combinational circuits implement essential functions of computers.
- But they are state-less: they only depend on their input.
- They have no memory of the past.
- Sequential circuits: the current output depends on
  - not only the current input, but also
  - the history of inputs.

Flip-Flop

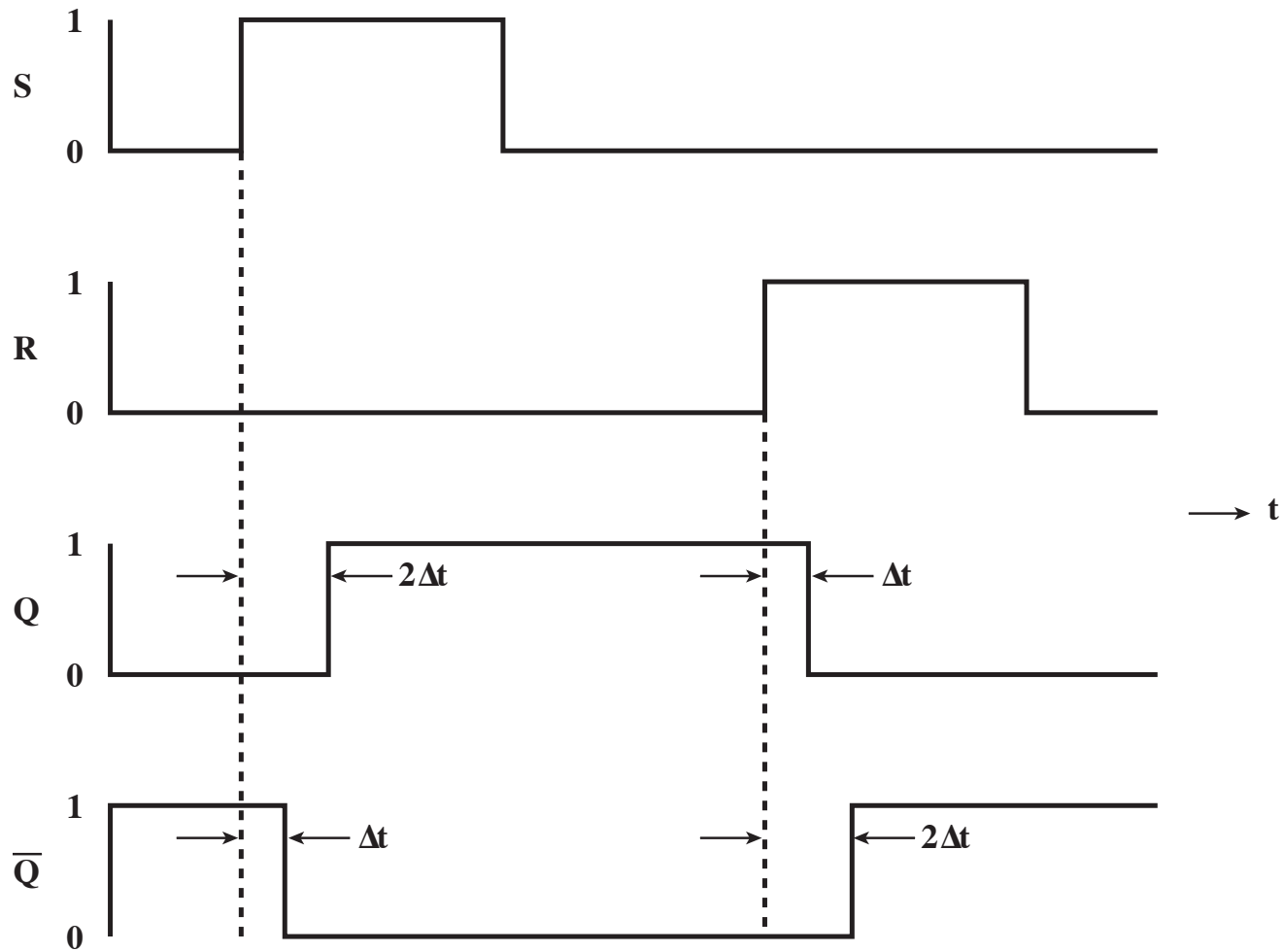
- The simplest form of sequential circuit.
- There are many variations of it.
- A bistable device: it has two stable states,
- so it functions as 1-bit memory!
- Usually there are two outputs which are complements of each other.

S-R Latch

- S: “set”, R: “reset”
- Here implemented with NOR gates.



S-R Latch



Characteristic Table for S-R Latch

- The input $SR = 11$ is not allowed: both Q and $\overline{Q}$ become 0.

SR	Q_n	$\boldsymbol{Q_{n+1}}$	$\overline{\boldsymbol{Q_{n+1}}}$
00	0	**0**	**1**
00	1	**1**	**0**
01	0	**0**	**1**
01	1	**0**	**1**
10	0	**1**	**0**
10	1	**1**	**0**
11	0	**0**	**0**
11	1	**0**	**0**

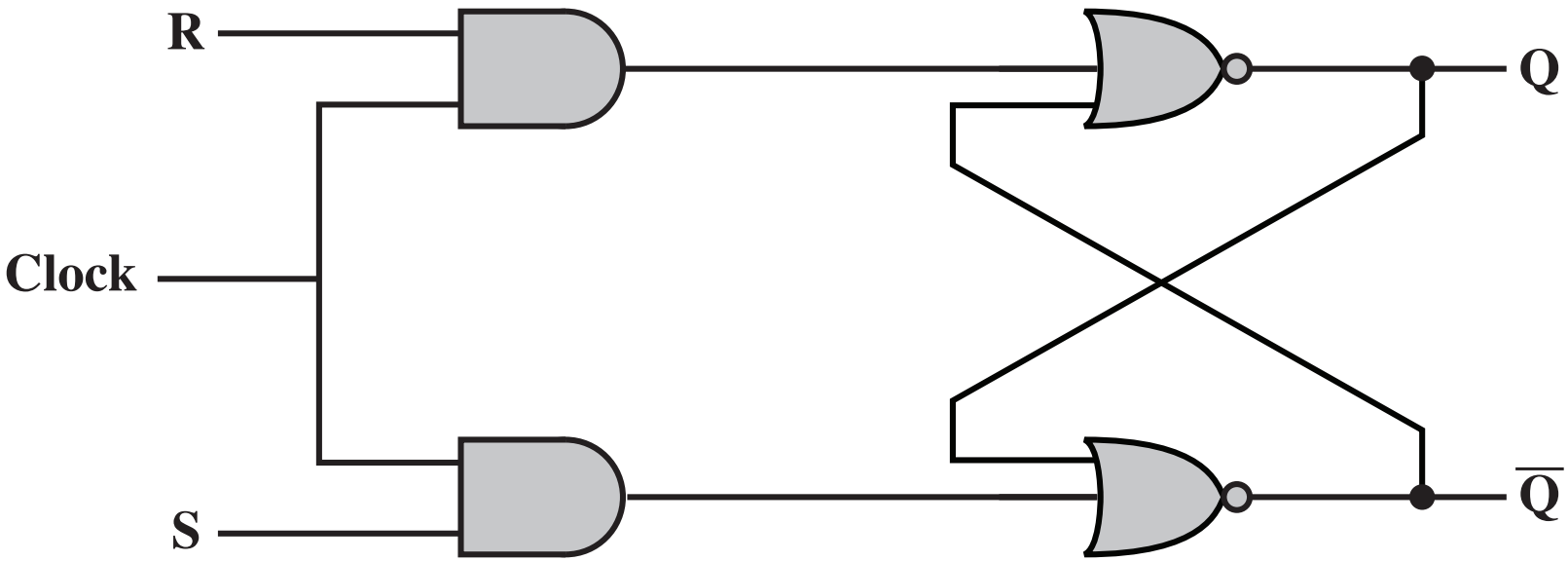
Simplified table:

S	R	$\boldsymbol{Q_{n+1}}$
0	0	$\boldsymbol{Q_n}$
0	1	**0**
1	0	**1**
1	1	—

Clocked S-R Flip-Flop

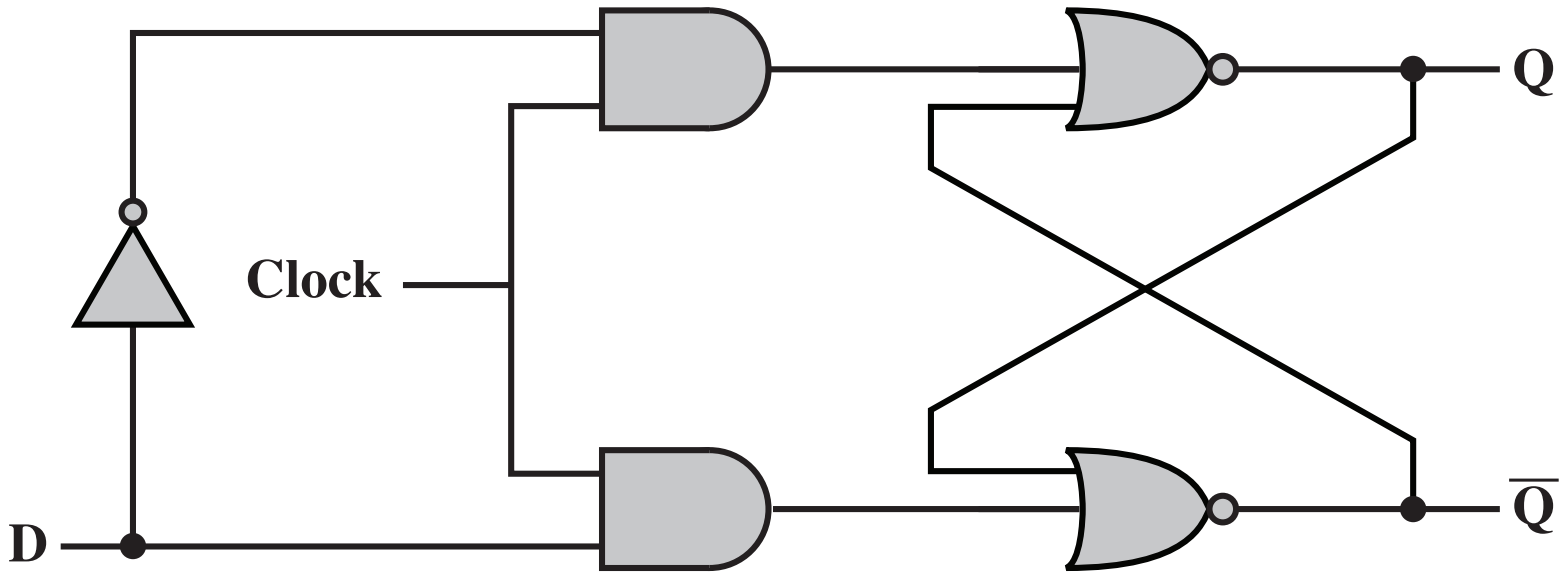
- S-R flip-flop is asynchronous: whenever the input changes the output will change.
- We need better control:

only change while the clock signal is 1.



D Flip-Flop

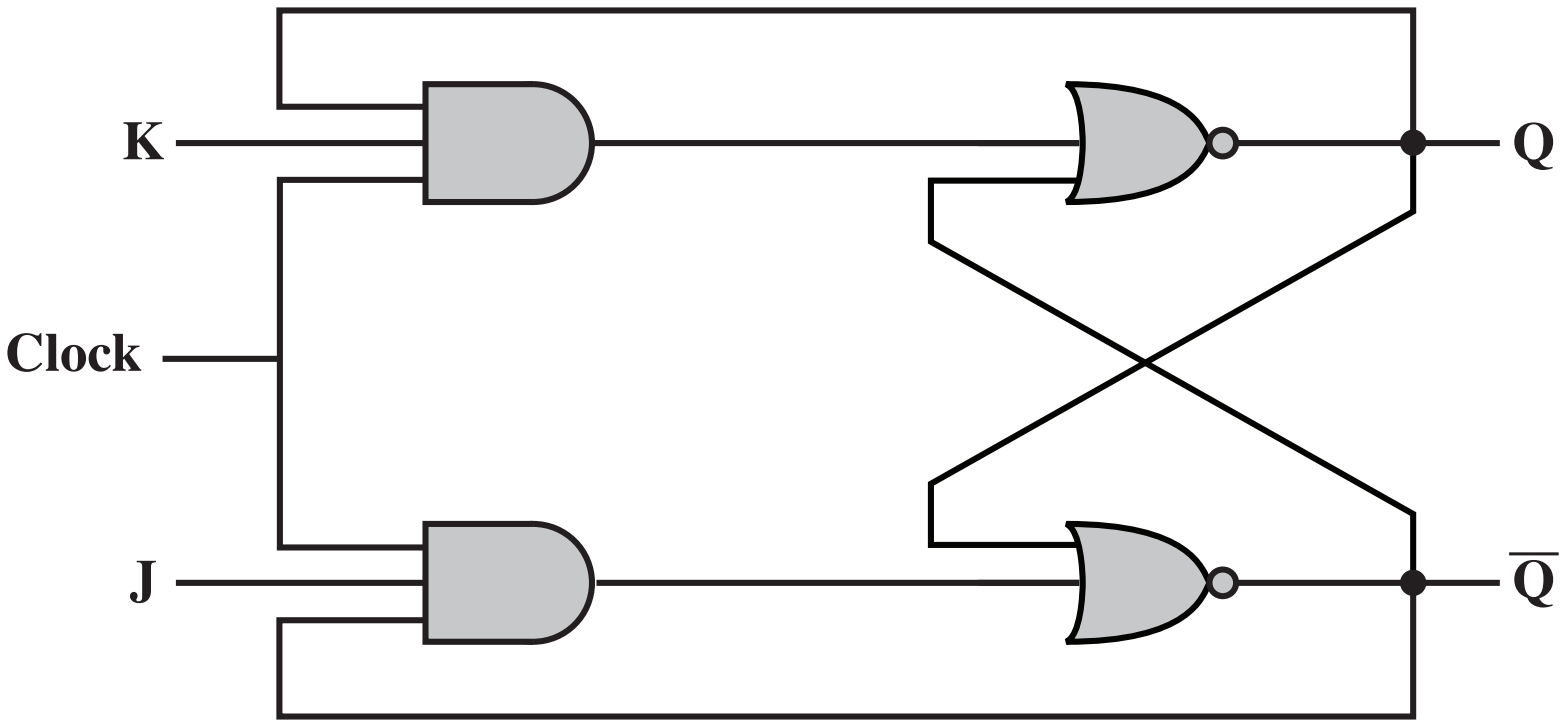
- Having a bad input state $SR = 11$ is icky.
- Often we any don't want to set or reset, we want to store a bit D.



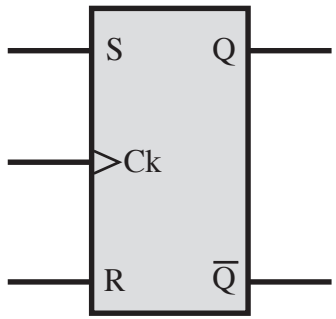
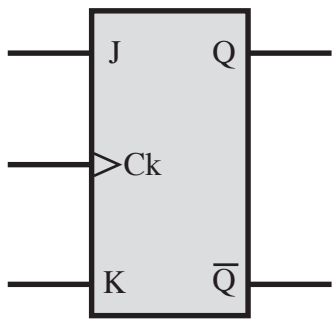
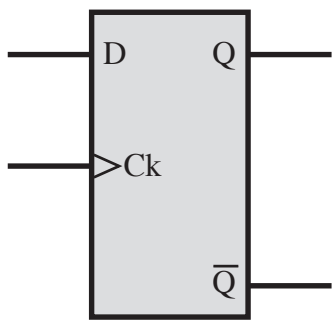
- Now we always have $R = \overline{S}$ (before the AND gates).
- Also known as a "data flip-flop".
- Also known as a "delay flip-flop": the input becomes the output only in the next clock cycle.

J-K Flip-Flop

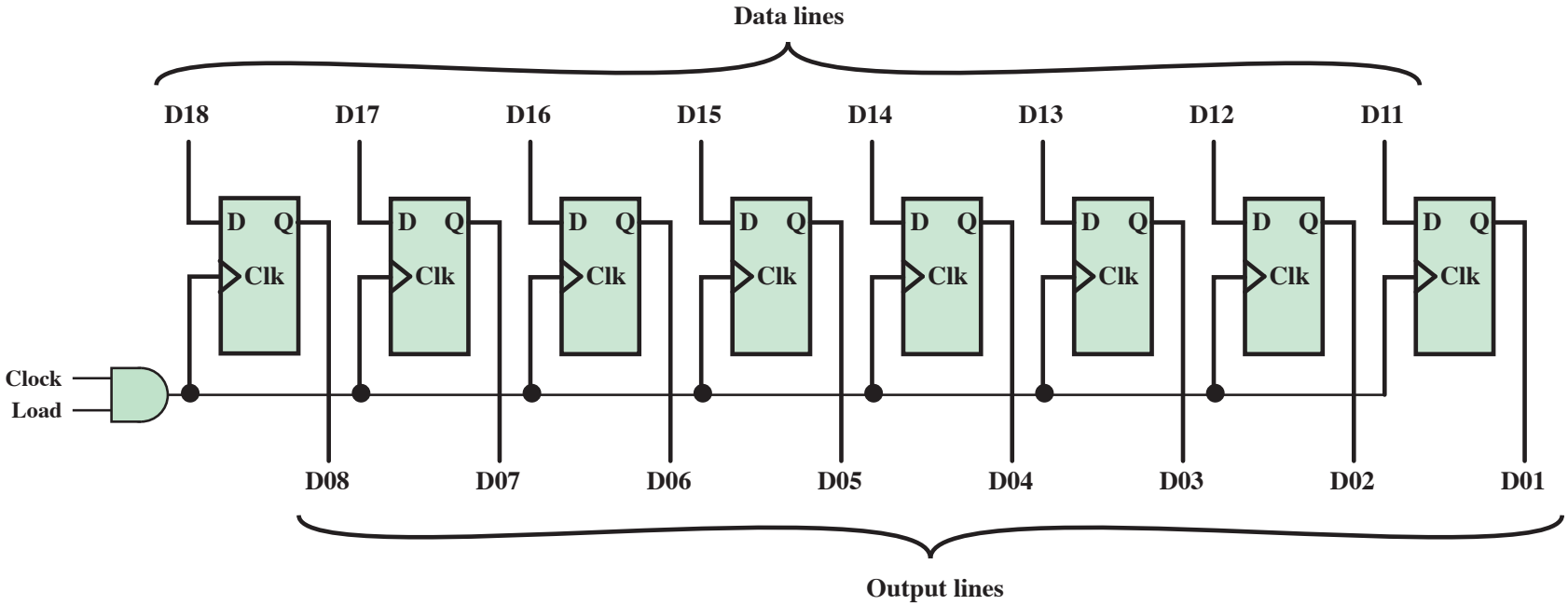
- A different modification of the clocked S-R Flip-Flop.
- Use the fourth input combination to toggle the output.
- When $JK = 11$ then $\overline{Q}Q$ goes through as the input.



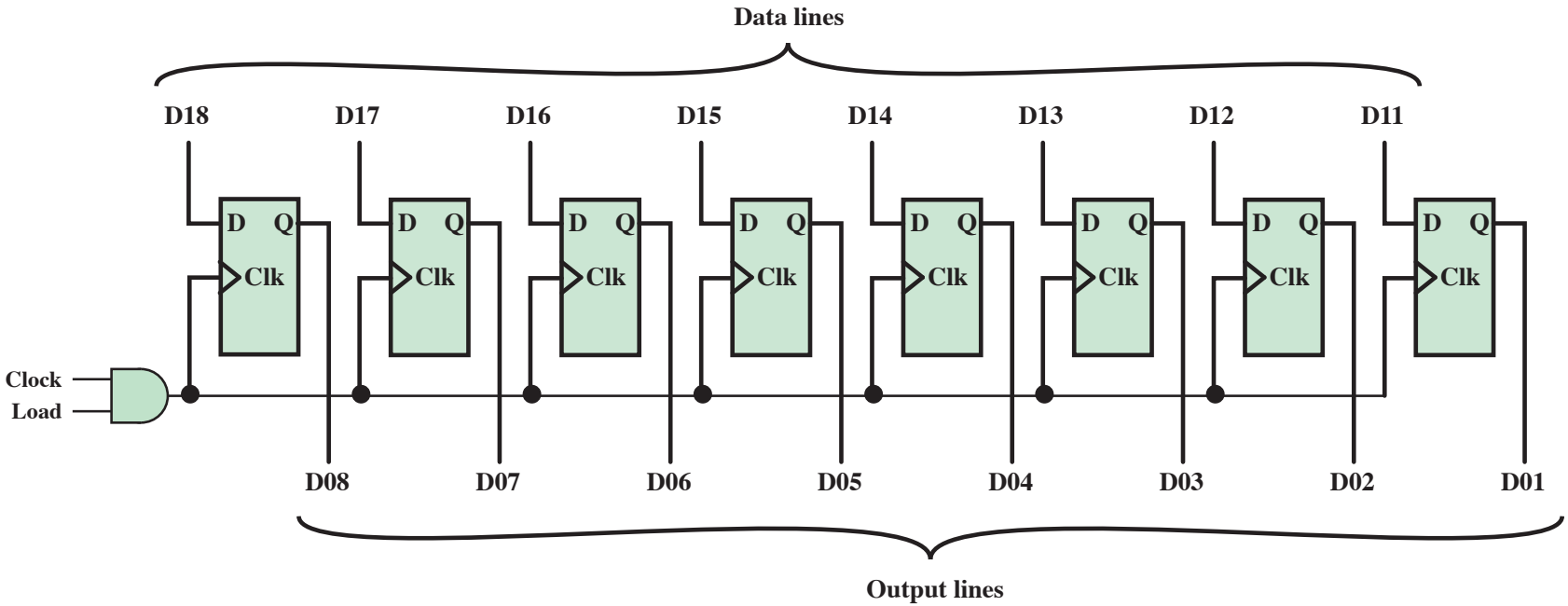
Flip-Flop Overview

Name	Graphical Symbol	Truth Table
S-R		S R Q_{n+1} 0 0 Q_n 0 1 0 1 0 1 1 1 –
J-K		J K Q_{n+1} 0 0 Q_n 0 1 0 1 0 1 1 1 $\overline{Q_n}$
D		D Q_{n+1} 0 0 1 1

Parallel Register



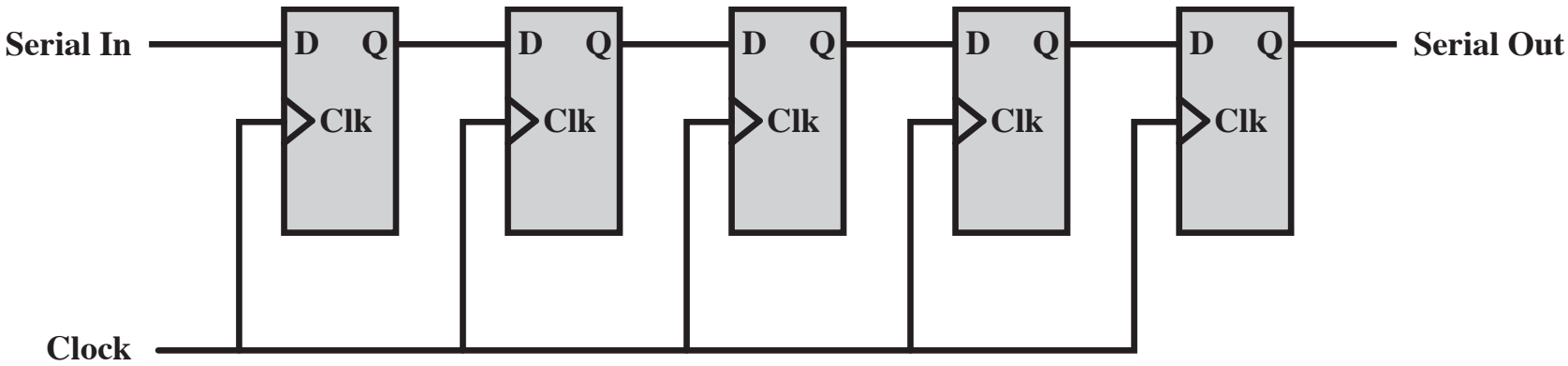
Parallel Register



Conceptually, how could `movq %rax, %rbx` be implemented?

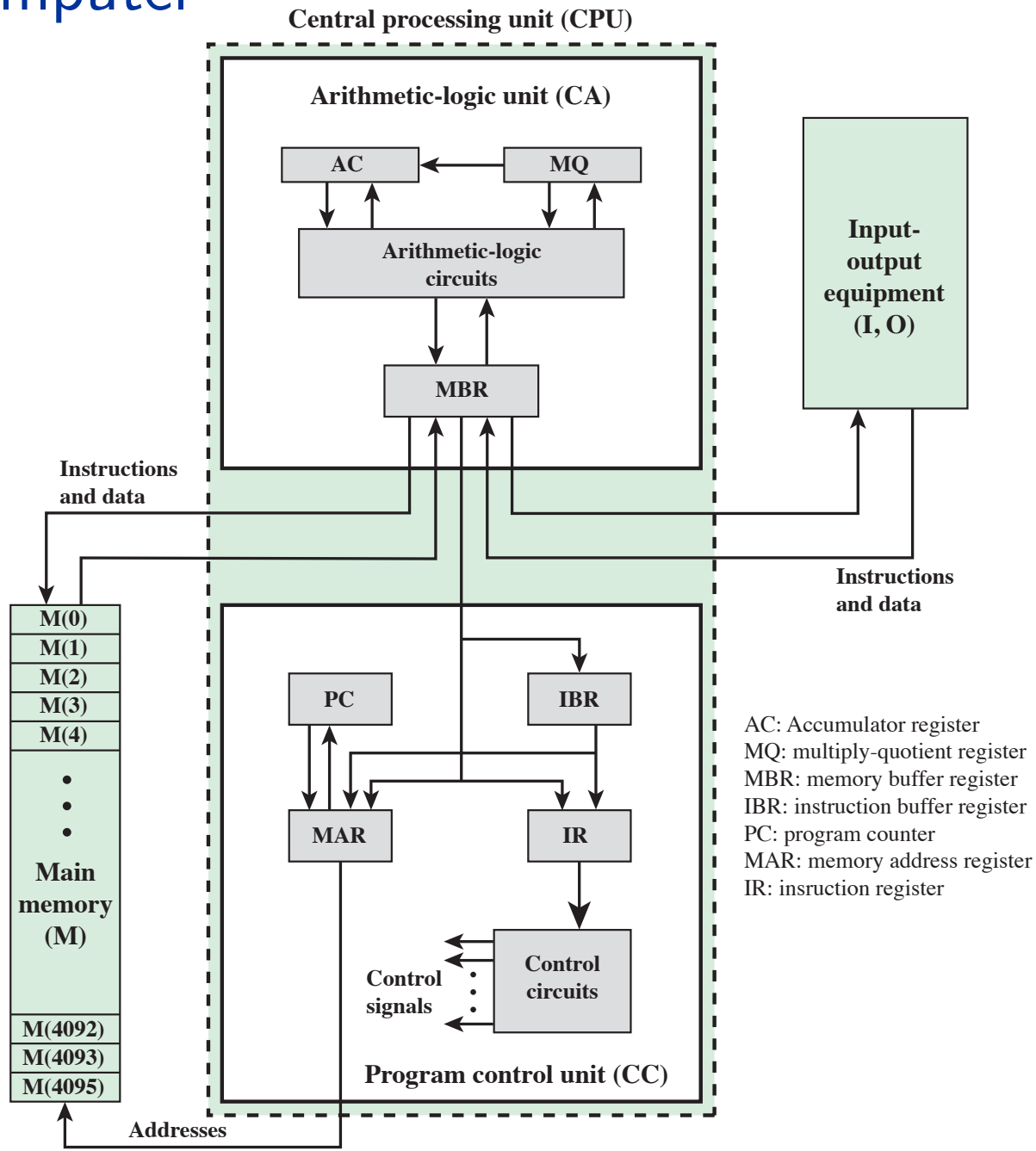
- The output from all registers could be connected to a multiplexer.
- The output of that multiplexer could be connected to the input of all registers.
- Select RAX as the output.
- Set "load" to 1 on the RBX register.
- Wait a clock cycle.

Shift Register



- Remember the multiplication and division algorithms?
- We could have a combined parallel and shift register,
- with an ALU and "slightly" more control logic.

The IAS Computer



Abstraction Levels